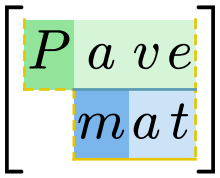
nicematrix for Typst

Dotted leaders, blocks, rules, fills, paths, braces and sub-matrices
for the matrices of Typst

$$A = \begin{array}{c} \begin{matrix} C_1 & \cdots & C_n \end{matrix} \\ \begin{bmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{m1} & \cdots & a_{mn} \end{bmatrix} \end{array} \begin{matrix} L_1 \\ \vdots \\ L_m \end{matrix}$$



The package `clanker-slop-nicematrix` merges the mathematical side of the LaTeX package nicematrix and the paths and fills of the Typst package pavemat.

Version 0.1.0 — by akad — https://github.com/coder56765/clanker-slop-nicematrix

Contents

I. Getting started 3
1 Introduction 3
2 Installation 3
3 A first matrix 3
4 A guided tour 4
5 Conventions 5
II. User guide 6
1 Dotted leaders 6
2 Arrows and curves 8
3 Cells and blocks 8
4 Rules and corners 9
5 Backgrounds 10
6 Paths along the grid 11
7 Exterior rows and columns 13
8 Braces and sub-matrices 14
9 Stacks of matrices 15
10 Inline equations 16
11 Every `mat` at once, and defaults 16
12 Drawing with the geometry 17
13 Named matrices 17
14 Showcase 19
III. Reference 21
1 `nicemat` and `nicearray` 21
2 Leaders 22
3 Markers 22
4 Functions 23
5 Geometry 23
6 Differences with nicematrix and pavemat 23
IV. The nicematrix manual, example by example 24
1 The vertical space between the rows 24
2 The vertical position of the arrays 24
3 The blocks 26
4 The rules 27
5 The color of the background of the rows and columns 30
6 The width of the columns 30
7 The exterior rows and columns 32
8 Dotted leaders in the matrices 33
9 Delimiters in the preamble of the environment 39
10 The \CodeAfter 39
11 Other features 43
12 Explicit use of TikZ with nicematrix 46
13 Technical remarks 48
14 Examples 49
15 Examples not ported 57

I. Getting started

1 Introduction

A `nicemat` is written like Typst's `mat`: `,` between the cells, `;` between the rows. Without any of the new features, it is laid out **exactly** like `mat` (same size, same delimiters, same cells), so it can replace it anywhere. On top of that, it provides:

- **dotted leaders**: the symbols `dots.c`, `dots.v`, `dots.down`… stretch between the cells, as the `\Cdots`, `\Vdots`… of nicematrix;
- **blocks**: cells spanning several rows and columns;
- **rules**: `hlines`, `vlines`, `augment`, and rules written among the cells, which respect the blocks, the leaders and the empty corners;
- **backgrounds**: fills by function, rectangles of cells, flood fills;
- **paths** drawn along the lines of the grid (from pavemat);
- **exterior rows and columns** for labels, **braces**, **sub-matrices**;
- **arrows** and curves between cells, even of different matrices;
- and access to the geometry of the matrix to draw anything on top of it.

The tabular side of nicematrix (`{NiceTabular}`, notes, captions…) is not covered: Typst's `table` already does most of it.

2 Installation

The package is on Typst Universe as `clanker-slop-nicematrix`; import it with:

```
#import "@preview/clanker-slop-nicematrix:0.1.0": *
```

Everything is exported under short names that work in math: `nicemat`, `nicearray`, the leaders `cdots`, `vdots`…, and the markers `cell`, `hline`, `submatrix`, `pave`… The source is at https://github.com/coder56765/clanker-slop-nicematrix.

One show rule is recommended at the top of the document, so that the matrices of inline equations get the small cells of an inline `mat` (see Section 10):

```
#show: nicemat-setup
```

3 A first matrix

Dots that stretch

```
$ A = nicemat(delim: "[",
  a_11, a_12, dots.c, a_(1 n);
  a_21, a_22, dots.c, a_(2 n);
  dots.v, dots.v, dots.down, dots.v;
  a_(m 1), a_(m 2), dots.c, a_(m n)) $
```

$$A = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix}$$

The dots are ordinary `dots.c`, `dots.v` and `dots.down`; inside a `nicemat` they stretch until they meet a cell.

One dot command per line

```
$ I_n = nicemat(
  1, 0, dots.c, 0;
  0, 1, , dots.v;
  dots.v, , dots.down, 0;
  0, dots.c, 0, 1) $
```

$$I_n = \begin{pmatrix} 1 & 0 & \cdots & 0 \\ 0 & 1 & & \vdots \\ \vdots & & \ddots & 0 \\ 0 & \cdots & 0 & 1 \end{pmatrix}$$

Try: leave another cell empty (just `,`) and see the leader grow through it.

4 A guided tour

Gaussian elimination

```
$ nicemat(last-col: #true, vlines: -1,
  align: #right,
  map-cells: #((i, j, it) => if j == 4 {
    text(blue, $script(it)$) } else { it }),
  2, 1, -1, 8, ;
  0, 1/2, 1/2, 1, L_2 <- L_2 + 3/2 L_1;
  0, 0, -1, 1, L_3 <- L_3 - 4 L_2) $
```

$$\left(\begin{array}{ccc|c} 2 & 1 & -1 & 8 \\ 0 & \frac{1}{2} & \frac{1}{2} & 1 \\ 0 & 0 & -1 & 1 \end{array}\right) \begin{array}{l} \\ L_2 \leftarrow L_2 + \frac{3}{2} L_1 \\ L_3 \leftarrow L_3 - 4 L_2 \end{array}$$

The row operations are written in an exterior column (`last-col`); `vlines: -1` is the last rule inside the matrix. Try: `vlines: #true`, or add `hlines: #true`.

Blocks and braces

```
$ M = nicemat(delim: "[",
  first-row: #true, last-col: #true,
  hbrace(k, span: #2), hbrace(n - k, span: #2);
  cell(A, rowspan: #2, colspan: #2,
    fill: #blue.lighten(85%)), 0, 0,
    vbrace(k, span: #2);
  0, 0;
  0, 0, cell(B, rowspan: #2, colspan: #2,
    fill: #red.lighten(85%)),
    vbrace(n - k, span: #2);
  0, 0) $
```

$$M = \begin{array}{c} \overbrace{}^{k} \; \overbrace{}^{n-k} \\ \left[\begin{array}{cc} A & \begin{matrix} 0 & 0 \\ 0 & 0 \end{matrix} \\ \begin{matrix} 0 & 0 \\ 0 & 0 \end{matrix} & B \end{array}\right] \end{array} \begin{array}{l} \}\, k \\ \}\, n-k \end{array}$$

Each block is one `cell` spanning several rows and columns; the positions it covers are skipped. Try: add `stroke: #blue` to a `cell(..)`.

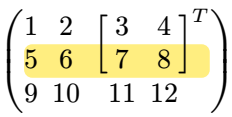
A Jacobian with labels

```
$ J_f = nicemat(first-row: #true,
  first-col: #true,
  map-cells: #((i, j, it) =>
    if i == 0 or j == 0 {
      text(gray.darken(40%), it) } else { it }),
  , x_1, dots.c, x_n;
  f_1, (partial f_1)/(partial x_1), dots.c,
    (partial f_1)/(partial x_n);
  dots.v, dots.v, dots.down, dots.v;
  f_m, (partial f_m)/(partial x_1), dots.c,
    (partial f_m)/(partial x_n)) $
```

$$J_f = \begin{array}{c} \\ f_1 \\ \vdots \\ f_m \end{array} \begin{array}{c} \begin{array}{cccc} x_1 & & \cdots & x_n \end{array} \\ \begin{pmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{pmatrix} \end{array}$$

Highlighting and sub-matrices

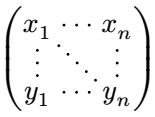
```
$ nicemat(
  1, 2, 3, 4;
  5, 6, 7, 8;
  9, 10, 11, 12,
  region((1, 0), (1, 3),
    fill: #yellow.lighten(50%), radius: #3pt),
  submatrix((0, 2), (1, 3), delim: "[",
    sup: T)) $
```



Try: move the region to another row, or change its `radius`.

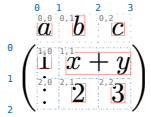
Every `mat` becomes nice

```
#show math.mat: nicemat
$ mat(x_1, dots.c, x_n;
  dots.v, dots.down, dots.v;
  y_1, dots.c, y_n) $
```



Where is what?

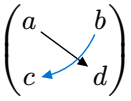
```
$ nicemat(debug: #true, first-row: #true,
  a, b, c;
  1, cell(x + y, colspan: #2);
  dots.v, 2, 3) $
```



`debug` shows the position of every cell (for `region`, `submatrix`, `hbrace`...) and the numbers of the rules, on the left and above (for `hlines`, `vlines`, `augment`, `pave`). `debug: "cells"` and `debug: "rules"` show only one of them.

Arrows between cells

```
$ nicemat(column-gap: #2em, row-gap: #1em,
  a, b;
  c, d,
  connect((0, 0), (1, 1)),
  connect((0, 1), (1, 0), bend: 30,
    stroke: #blue)) $
```



`connect` draws an arrow, straight or bent (like TikZ's `bend left`; negative angles bend to the right).

5 Conventions

Code values in math a function called in math receives math content, so code values need a `#`: `#true`, `#red`, `#2pt`, `#(dash: "dashed")`. Small integers (`span: 3`), coordinates (`(0, 2)`) and lists of indices (`hlines: (1, -1)`) work without it.

Aliases a function used in math needs a name of two letters or more: `#let mm = nicemat.with(..)` works as `$mm(..)$`, a one-letter name doesn't.

Cells they are addressed `(row, col)`, from 0, **as written**, exterior rows and columns included. Negative numbers count from the end.

Rules and paths they only exist inside the main block and are counted within it, like the `augment` of `mat`: the rule 0 is the top (left) border. A grid point `(i, j)` of a path is at the crossing of the rules `i` and `j`.

Spans a spanning cell, leader or brace skips the positions it covers, as in `table`: don't write empty placeholders under it.

Rectangles `region`, `submatrix` and the braces take two corners `(from, to)`, or whole rows and columns: `row: 1`, `rows: (0, 2)`, `col: 3`, `cols: (1, 2)`; the other direction is then the whole matrix (without its exterior rows and columns).

Outside a `nicemat` the markers still show something sensible (a leader its dots, a `cell` its body, a `diagbox` x/y), so a matrix stays readable in a plain `mat`.

Defaults `#let nicemat = nicemat.with(..)` or `#show math.mat: nicemat.with(..)`; the `set math.mat(..)` rules (`delim`, `align`, `gap`, `row-gap`, `column-gap`) are honoured.

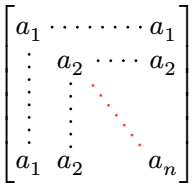
II. User guide

1 Dotted leaders

In a `nicemat`, the symbols `dots.h` (or `...`), `dots.c`, `dots.v`, `dots.down` and `dots.up` are drawn as leaders between the nearest non-empty cells on both sides. The empty cells in between are crossed, and consecutive dots of the same kind make a single line. The functions `ldots`, `cdots`, `vdots`, `ddots` and `iddots` give the same leaders with options; the option `dots` of `nicemat` sets them for all the leaders.

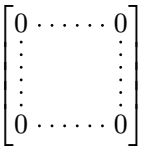
Leaders between non-empty cells

```
$ nicemat(delim: "[",
  a_1, dots.c, , , a_1;
  dots.v, a_2, dots.c, , a_2;
  , dots.v, ddots(fill: #red);
  ;
  a_1, a_2, , , a_n) $
```



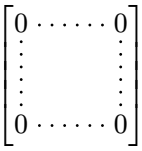
The null matrix

```
$ nicemat(delim: "[",
  0, dots.c, dots.c, 0;
  dots.v, , , dots.v;
  dots.v, , , dots.v;
  0, dots.c, dots.c, 0) $
```



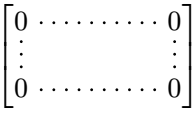
Same matrix, fewer dots

```
$ nicemat(delim: "[",
  0, dots.c, , 0;
  dots.v, , , ;
  , , , dots.v;
  0, , dots.c, 0) $
```



A wider matrix

```
$ nicemat(delim: "[",
  0, dots.c, #h(1cm), 0;
  dots.v, , , dots.v;
  0, dots.c, , 0) $
```



A cell holding only `#h(..)` counts as empty (nicematrix's `\Hspace`).

Leaders that take no room

```
$ C = nicemat(h, i, j, k, l, m;
  x, dots.h, dots.h, dots.h, dots.h, x) $
$ D = nicemat(dots: #(nullify: true),
  h, i, j, k, l, m;
  x, dots.h, , , , x) $
```

$$C = \begin{pmatrix} h & i & j & k & l & m \\ x & \cdots & \cdots & \cdots & \cdots & x \end{pmatrix}$$

$$D = \begin{pmatrix} h & i & j & k & l & m \\ x & \cdots & \cdots & \cdots & \cdots & x \end{pmatrix}$$

Leaders over several columns

```
$ nicemat(1, 2, 3, 4, 5;
  1, cdots(span: #3), 5;
  1, 2, 3, 4, 5)
  quad
  nicemat(1, 2, 3, 4, 5;
  , cdots(span: #3);
  1, 2, 3, 4, 5) $
```

$$\begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 1 & \cdots & \cdots & \cdots & 5 \\ 1 & 2 & 3 & 4 & 5 \end{pmatrix} \quad \begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ & \cdots & \cdots & \cdots & \\ 1 & 2 & 3 & 4 & 5 \end{pmatrix}$$

With `span`, a leader covers several columns (like `\Hdotsfor`) or rows (`vdots(span: ..)`, like `\Vdotsfor`) and only looks at its neighbours.

Labels

```
$ nicemat(delim: "[",
  1, #h(1cm), 0;
  , ddots(above: n "times"), ;
  0, , 1)
  quad
  nicemat(delim: "[",
  1, #h(1cm), 0;
  , ddots(above: n "times",
    horizontal-labels: #true), ;
  0, , 1) $
```

$$\begin{bmatrix} 1 & \cdots & 0 \\ & \overset{n \text{ times}}{\ddots} & \\ 0 & & 1 \end{bmatrix} \quad \begin{bmatrix} 1 & \cdots & 0 \\ & \overset{n \text{ times}}{\ddots} & \\ 0 & & 1 \end{bmatrix}$$

The labels are `above`, `below` and `middle` (nicematrix's `^`, `_` and `:`).

Other line styles

```
$ nicemat(dots: #(nullify: true,
    stroke: (dash: "loosely-dotted")),
  a, b, 0, , dots.c, 0;
  b, a, b, dots.down, , dots.v;
  0, b, a, dots.down, , ;
  , dots.down, dots.down, dots.down, , 0;
  dots.v, , , , , b;
  0, dots.c, , 0, b, a) $
```

$$\begin{pmatrix} a & b & 0 & \cdots & 0 \\ b & a & b & \ddots & \vdots \\ 0 & b & a & \ddots & 0 \\ \vdots & \ddots & \ddots & \ddots & b \\ 0 & \cdots & 0 & b & a \end{pmatrix}$$

`stroke: auto` gives round dots; any stroke gives a line, which can get arrow tips with `marks`.

Parallel diagonals

```
$ A = nicemat(
  1, dots.c, , 1;
  a+b, dots.down, , dots.v;
  dots.v, dots.down, , ;
  a+b, dots.c, a+b, 1) $
```

$$A = \begin{pmatrix} 1 & \cdots & \cdots & 1 \\ a+b & \ddots & & \vdots \\ \vdots & \ddots & \ddots & \vdots \\ a+b & \cdots & a+b & 1 \end{pmatrix}$$

Diagonal leaders are drawn parallel to the first one (or to the one marked `first: #true`); `dots: (parallel: false)` turns this off.

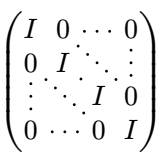
A linear system

```
$ nicemat(vlines: -1, last-col: #true,
  dots: #(nullify: true),
  map-cells: #((i, j, it) =>
    if j == 7 { $script(it)$ } else { it }),
  1, 1, 1, dots.c, , 1, 0, ;
  0, 1, 0, dots.c, , 0, , L_2 <- L_2 - L_1;
  0, 0, 1, dots.down, , dots.v, ,
    L_3 <- L_3 - L_1;
  , , , dots.down, , , dots.v, dots.v;
  dots.v, , , dots.down, , 0, ;
  0, , , dots.c, 0, 1, 0, L_n <- L_n - L_1) $
```

$$\left(\begin{array}{ccccc|c} 1 & 1 & 1 & \cdots & 1 & 0 \\ 0 & 1 & 0 & \cdots & 0 & \vdots \\ 0 & 0 & 1 & \ddots & \vdots & \vdots \\ \vdots & \vdots & \ddots & \ddots & 0 & \vdots \\ 0 & \cdots & \cdots & 0 & 1 & 0 \end{array}\right) \begin{array}{l} \\ L_2 \leftarrow L_2 - L_1 \\ L_3 \leftarrow L_3 - L_1 \\ \\ L_n \leftarrow L_n - L_1 \end{array}$$

A leader between any two cells

```
$ nicemat(
  I, 0, dots.c, 0;
  0, I, dots.down, dots.v;
  dots.v, dots.down, I, 0;
  0, dots.c, 0, I,
  dotline((1, 1), (2, 2))) $
```



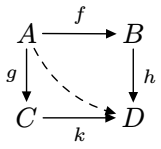
`dotline(from, to)` (nicematrix's `\line`) links two cells with the same options as the leaders.

2 Arrows and curves

`connect(from, to)` is a `dotline` drawn as an arrow: a solid line in the color of the text with an arrow head (`marks: "->"`). Both take `bend`, an angle (a plain number is in degrees, so math needs no `#`): the line leaves the first cell turned by that angle to its left and reaches the second one symmetrically, like TikZ's `bend left`; a negative angle bends to the right.

A commutative diagram

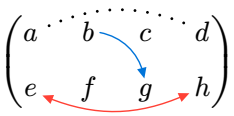
```
$ nicearray(column-gap: #3em, row-gap: #2em,
  A, B;
  C, D,
  connect((0, 0), (0, 1), above: f),
  connect((0, 0), (1, 0), below: g,
    horizontal-labels: #true),
  connect((0, 1), (1, 1), above: h,
    horizontal-labels: #true),
  connect((1, 0), (1, 1), below: k),
  connect((0, 0), (1, 1), bend: -25,
    stroke: #(dash: "dashed"))) $
```



The labels follow the line unless `horizontal-labels: #true`.

Bent leaders

```
$ nicemat(column-gap: #1.5em, row-gap: #1em,
  a, b, c, d;
  e, f, g, h,
  dotline((0, 0), (0, 3), bend: 25),
  connect((1, 0), (1, 3), bend: -25,
    stroke: #red, marks: "<->"),
  connect((0, 1), (1, 2), bend: 40,
    stroke: #blue)) $
```



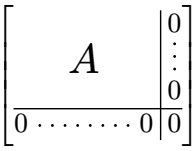
Dots are spaced evenly along a curve; `stroke`, `marks`, `shorten` and the labels work as for straight leaders, and the arrow head takes the color of the line.

3 Cells and blocks

`cell(body, ..)` gives options to a cell and lets it span several rows and columns (`\Block`). The positions it covers are skipped, as with `table.cell`. `diagbox(lower, upper)` slashes a cell diagonally.

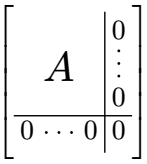
A block

```
$ nicemat(delim: "[", vlines: 3,
  column-gap: #(0.5em, 1cm, 0.5em),
  cell(#text(1.5em)[$A$],
    rowspan: #3, colspan: #3), 0;
  dots.v;
  0;
  hline();
  0, dots.c, 0, 0) $
```



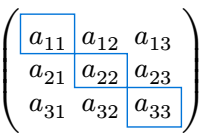
Rules skip blocks and leaders

```
$ nicemat(delim: "[", hlines: #true,
  vlines: #true, margin: #0.2em,
  cell(#text(1.5em)[$A$],
    rowspan: #3, colspan: #3), 0;
  dots.v;
  0;
  0, dots.c, 0, 0) $
```



Highlighted cells

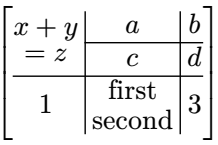
```
$ nicemat(margin: #0.3em, stroke: #blue,
  cell(a_11, stroke: #true), a_12, a_13;
  a_21, cell(a_22, stroke: #true), a_23;
  a_31, a_32, cell(a_33, stroke: #true)) $
```



`stroke: #true` uses the stroke of the rules.

Several lines in a cell

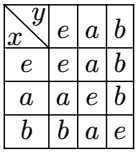
```
$ nicemat(delim: "[", hvlines: #true,
  cell(x + y \ = z, rowspan: #2), a, b;
  c, d;
  1, "first" \ "second", 3) $
```



`\` breaks a cell (or a block) into lines, set like the rows of a matrix and aligned like the cell.

Diagonal boxes

```
$ nicearray(hlines: #true, vlines: #true,
  diagbox(x, y), e, a, b;
  e, e, a, b;
  a, a, e, b;
  b, b, a, e) $
```

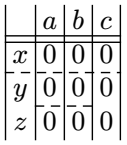


4 Rules and corners

The options `hlines` and `vlines` take `true` (all the rules inside, plus the borders without delimiters), an index or a list of indices; `hvlines: #true` gives both. `borders` forces the outer rules on (`#true`) or off (`#false`, nicematrix's `hvlines-except-borders`). `augment` works as in `mat`. Rules can also be written among the cells: `hline()` on its own row separates the neighbouring rows, `vline()` in a row is drawn after the cell it follows; `start` and `end` limit them, and two markers at the same place give a double rule.

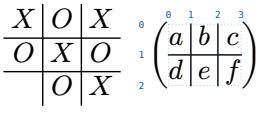
Rules as markers, dashed and double

```
$ nicearray(vlines: #true,
  , a, b, c;
  hline(); hline();
  x, 0, 0, 0;
  hline(stroke: #(dash: "dashed"));
  y, 0, 0, 0;
  hline(start: #1, end: #3,
    stroke: #(dash: "dashed"));
  z, 0, 0, 0) $
```



All the rules, without the borders

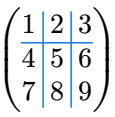
```
$ nicearray(hvlines: #true, borders: #false,
  X, O, X; O, X, O; , O, X)
  quad
  nicemat(hvlines: #true, debug: "rules",
  a, b, c; d, e, f) $
```



`debug: "rules"` shows the numbers of the rules, as `hlines`, `vlines`, `augment` and `pave` count them.

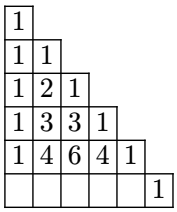
`augment`, as in `mat`

```
$ nicemat(1, 2, 3; 4, 5, 6; 7, 8, 9,
  augment: #(hline: 1, vline: (1, -1),
    stroke: blue)) $
```



Empty corners

```
$ nicearray(corners: #(top + right),
  hlines: #true, vlines: #true,
  1; 1, 1; 1, 2, 1; 1, 3, 3, 1;
  1, 4, 6, 4, 1; , , , , , 1) $
```



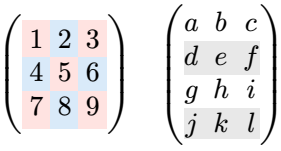
No rule is drawn in the empty corners; `corners: #true` gives the four of them, and `top`, `right` ... the generalized corners.

5 Backgrounds

`fill` takes a color, an array (cycled over the columns) or a function `(i, j) => color`; `region(from, to, ..)` covers any rectangle of cells (`region(row: 1, ..)`, `region(cols: (0, 1), ..)` whole rows or columns), and `flood(from, fill: ..)` fills the region bounded by paths (see the next chapter).

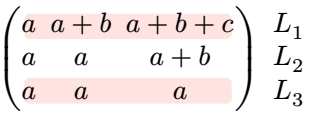
Fills

```
$ nicemat(margin: #0.25em,
  fill: #((i, j) => if calc.even(i + j) {
    red.lighten(85%) } else {
    blue.lighten(85%) }),
  1, 2, 3; 4, 5, 6; 7, 8, 9)
  quad
  nicemat(fill: #((i, j) =>
    if calc.odd(i) { luma(232) }),
  a, b, c; d, e, f; g, h, i; j, k, l) $
```



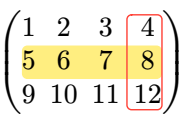
Regions

```
$ nicemat(last-col: #true,
  a, a + b, a + b + c, L_1;
  a, a, a + b, L_2;
  a, a, a, L_3,
  region((0, 0), (0, 2), outset: #(-1pt),
    fill: #red.lighten(85%), radius: #2pt),
  region((2, 0), (2, 2), outset: #(-1pt),
    fill: #red.lighten(85%), radius: #2pt)) $
```



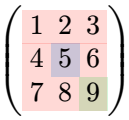
Whole rows and columns

```
$ nicemat(
  1, 2, 3, 4; 5, 6, 7, 8; 9, 10, 11, 12,
  region(row: 1, radius: #2pt,
    fill: #yellow.lighten(50%)),
  region(col: -1, stroke: #red, radius: #2pt)) $
```



Flood fills and single cells

```
$ nicemat(
  1, 2, 3; 4, 5, 6; 7, 8, 9,
  flood((0, 0), fill: #red.transparentize(80%)),
  region((1, 1), (1, 1),
    fill: #blue.transparentize(80%)),
  region((2, 2), (2, 2),
    fill: #green.transparentize(80%))) $
```



Without any path, the flood from `(0, 0)` covers the whole matrix; the regions of a single cell are drawn on top of it.

6 Paths along the grid

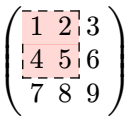
The paths of pavemat are drawn with `pave(path, ..)`. The path is a string of directions, starting from a grid point:

- `W`, `w`: up; `S`, `s`: down; `A`, `a`: left; `D`, `d`: right.

An upper-case letter draws the segment, a lower-case letter only moves. The grid point `(i, j)` is at the crossing of the rule `i` (above the row `i`) and of the rule `j` (left of the column `j`), counted within the matrix like the rules; the default is the top-left corner, and `from` takes a point or a corner (`top + right`...). The stroke of a path is the `stroke` of the matrix (pavemat draws dashed lines by default, so several examples below set `stroke: #(dash: "dashed", thickness: 0.5pt)`), and `pave(.., stroke: ..)` changes it for one path.

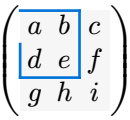
A path and a flood fill

```
$ nicemat(
  stroke: #(dash: "dashed", thickness: 0.5pt),
  1, 2, 3; 4, 5, 6; 7, 8, 9,
  pave("DDSSAAW"),
  flood((1, 1),
    fill: #red.transparentize(80%))) $
```



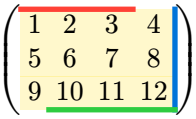
Drawing paths through data

```
$ nicemat(stroke: #(blue + 1pt),
  fill: #gray.transparentize(90%),
  a, b, c; d, e, f; g, h, i,
  pave("DDSSAAW")) $
```



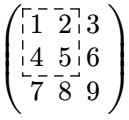
Several styled paths

```
$ nicemat(
  1, 2, 3, 4; 5, 6, 7, 8; 9, 10, 11, 12,
  pave("DDD", stroke: #(red + 2pt)),
  pave("SSS", from: #(top + right),
    stroke: #(blue + 2pt)),
  pave("AAA", from: #(bottom + right),
    stroke: #(green + 2pt)),
  flood((1, 1),
    fill: #yellow.transparentize(80%))) $
```



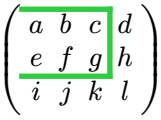
The border of the matrix

```
$ nicemat(
  stroke: #(dash: "dashed", thickness: 0.5pt),
  1, 2, 3; 4, 5, 6; 7, 8, 9,
  pave("DDSSAAWW")) $
```



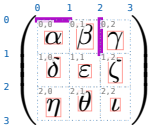
A flow from top-left to bottom-right

```
$ nicemat(stroke: #(paint: green, thickness:
2pt),
  a, b, c, d; e, f, g, h; i, j, k, l,
  pave("DDDSSAAA")) $
```



Hidden segments

```
$ nicemat(stroke: #(purple + 2pt),
  debug: #true,
  alpha, beta, gamma;
  delta, epsilon, zeta;
  eta, theta, iota,
  pave("DdSs")) $
```



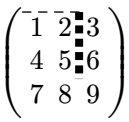
Lower-case letters move without drawing; `debug` shows the hidden segments in gray. Hidden segments still separate the regions of `flood`.

6.1 Styled segments

A group in parentheses changes the stroke of the following segments; it is evaluated as a dictionary of stroke properties, added to the current stroke. Groups can be nested, and `]` goes back to the previous stroke (a `[` after the group is allowed and ignored). Single quotes are replaced by double quotes, so there is nothing to escape.

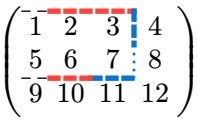
A thick dotted part

```
$ nicemat(
  stroke: #(dash: "dashed", thickness: 0.5pt),
  1, 2, 3; 4, 5, 6; 7, 8, 9,
  pave("DD(thickness: 3pt, dash: 'dotted')SS"))
$
```



Nested styles

```
$ nicemat(
  stroke: #(dash: "dashed", thickness: 0.5pt),
  1, 2, 3, 4; 5, 6, 7, 8; 9, 10, 11, 12,
  pave("D(paint: red, thickness: 2pt)DD(paint: blue)S(thickness: 1pt, dash: 'dotted')S]A]A]A")) $
```

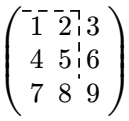


`D` moves right from $(0,0)$ to $(0,1)$; `(paint: red, thickness: 2pt)DD` draws two red segments 2pt thick up to $(0,3)$; `(paint: blue)S` goes down in blue (still 2pt) to $(1,3)$; `(thickness: 1pt, dash: 'dotted')S` goes down to $(2,3)$ in a dotted blue line of 1pt; each `]` then goes back one style: the last three segments `A` are blue 2pt, red 2pt and finally the dashed stroke of the matrix.

6.2 Other letters and starting points

`UDLR` instead of `WASD`

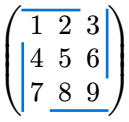
```
$ nicemat(
  stroke: #(dash: "dashed", thickness: 0.5pt),
  1, 2, 3; 4, 5, 6; 7, 8, 9,
  pave("RRDDUULL", chars: #(up: "U",
    down: "D", left: "L", right: "R"))) $
```



Be careful with the case: a lower-case letter is a hidden segment.

Starting from the corners

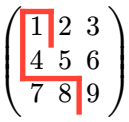
```
$ nicemat(stroke: #(blue + 1pt),
  1, 2, 3; 4, 5, 6; 7, 8, 9,
  pave("DD"),
  pave("SS", from: #(top + right)),
  pave("AA", from: #(bottom + right)),
  pave("WW", from: #(bottom + left))) $
```



The grid points go from $(0,0)$ to (n,m) for a matrix of n rows and m columns, whose cells go from $(0,0)$ to $(n-1,m-1)$.

Starting from a grid point

```
$ nicemat(stroke: #(red + 2pt),
  1, 2, 3; 4, 5, 6; 7, 8, 9,
  pave("WASSDDS", from: (1, 1))) $
```

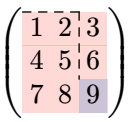


6.3 Flood fills

`flood(from, fill: ..)` fills the cells connected to the cell `from` without crossing a path (drawn or hidden); `region((i, j), (i, j), fill: ..)` fills a single cell. The fills are drawn in the order of the arguments, after the `fill` of the matrix: later ones cover earlier ones.

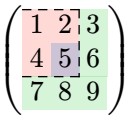
A region and a single cell

```
$ nicemat(
  stroke: #(dash: "dashed", thickness: 0.5pt),
  1, 2, 3; 4, 5, 6; 7, 8, 9,
  pave("DDSS"),
  flood((0, 0), fill: #red.transparentize(80%)),
  region((2, 2), (2, 2),
    fill: #blue.transparentize(80%))) $
```



Connected regions

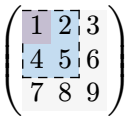
```
$ nicemat(
  stroke: #(dash: "dashed", thickness: 0.5pt),
  1, 2, 3; 4, 5, 6; 7, 8, 9,
  pave("DDSSAA"),
  flood((0, 0), fill: #red.transparentize(80%)),
  region((1, 1), (1, 1),
    fill: #blue.transparentize(80%)),
  flood((-1, -1),
    fill: #green.transparentize(80%))) $
```



Negative coordinates count from the end: `(-1, -1)` is the bottom-right cell.

A global fill with overrides

```
$ nicemat(
  stroke: #(dash: "dashed", thickness: 0.5pt),
  fill: #gray.transparentize(90%),
  1, 2, 3; 4, 5, 6; 7, 8, 9,
  pave("DDSSAAW"),
  region((0, 0), (0, 0),
    fill: #red.transparentize(80%)),
  flood((1, 1),
    fill: #blue.transparentize(80%))) $
```

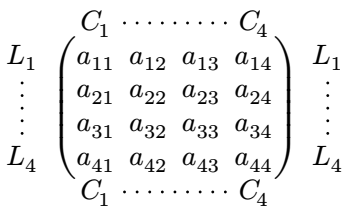


7 Exterior rows and columns

`first-row`, `last-row`, `first-col` and `last-col` put the first or last row or column outside the delimiters. `map-cells: (i, j, body) => body` styles any cell (nicematrix's `code-for-first-row`, `\RowStyle`…).

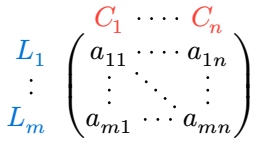
Labels around the matrix

```
$ nicemat(first-row: #true, last-row: #true,
  first-col: #true, last-col: #true,
  dots: #(nullify: true),
  , C_1, dots.c, , C_4, ;
  L_1, a_11, a_12, a_13, a_14, L_1;
  dots.v, a_21, a_22, a_23, a_24, dots.v;
  , a_31, a_32, a_33, a_34, ;
  L_4, a_41, a_42, a_43, a_44, L_4;
  , C_1, dots.c, , C_4, ) $
```



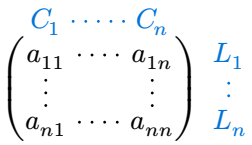
Styling the exterior

```
$ nicemat(first-row: #true, first-col: #true,
  map-cells: #((i, j, it) =>
    if i == 0 { text(red, it) }
    else if j == 0 { text(blue, it) }
    else { it }),
  , C_1, dots.c, C_n;
  L_1, a_11, dots.c, a_(1 n);
  dots.v, dots.v, dots.down, dots.v;
  L_m, a_(m 1), dots.c, a_(m n)) $
```



Colored leaders in the exterior

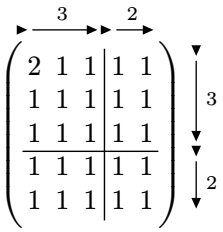
```
$ nicemat(first-row: #true, last-col: #true,
  map-cells: #((i, j, it) =>
    if i == 0 or j == 4 { text(blue, it) }
    else { it }),
  dots: #(fill: (i, j) =>
    if i == 0 or j == 4 { blue }),
  C_1, dots.c, , C_n, ;
  a_11, dots.c, , a_(1 n), L_1;
  dots.v, , , dots.v, dots.v;
  a_(n 1), dots.c, , a_(n n), L_n) $
```



`fill` of the leaders can be a function of the cell where the leader starts (`none` keeps the color of the text), to follow `map-cells`.

Counting rows and columns

```
$ nicemat(first-row: #true, last-col: #true,
  margin: #0.15em,
  dots: #(stroke: 0.5pt, marks: "<->",
    horizontal-labels: true),
  cdots(span: #3, above: 3),
    cdots(span: #2, above: 2);
  2, 1, 1, vline(), 1, 1,
    vdots(span: #3, above: 3);
  1, 1, 1, 1, 1;
  1, 1, 1, 1, 1;
  hline();
  1, 1, 1, 1, 1, vdots(span: #2, above: 2);
  1, 1, 1, 1, 1) $
```

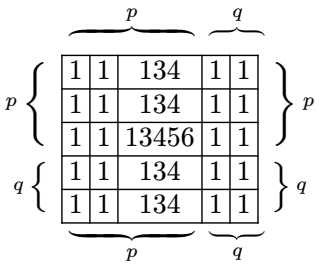


8 Braces and sub-matrices

In the exterior rows and columns, `hbrace(label, span: n)` and `vbrace(label, span: n)` draw braces over `n` columns or rows (`\Hbrace`, `\Vbrace`). Anywhere, `hbrace(from, to, label)` and `vbrace(from, to, label)` draw a brace along a rectangle of cells (`\OverBrace`, `\UnderBrace`), and `submatrix(from, to, ..)` puts delimiters around it (`\SubMatrix`). They make room for themselves. Instead of `from, to`, they take whole rows and columns: `hbrace(A, cols: (0, 2))`, `vbrace(B, rows: (1, 2))`, `submatrix(rows: (0, 1))`.

Braces in the exterior rows

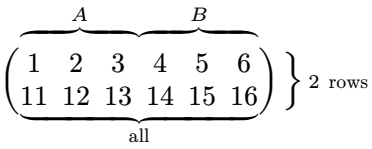
```
$ nicearray(first-row: #true,
  last-row: #true, first-col: #true,
  last-col: #true, hlines: #true,
  vlines: #true,
  , hbrace(p, span: #3), hbrace(q, span: #2);
  vbrace(p, span: #3), 1, 1, 134, 1, 1,
    vbrace(p, span: #3);
  1, 1, 134, 1, 1;
  1, 1, 13456, 1, 1;
  vbrace(q, span: #2), 1, 1, 134, 1, 1,
    vbrace(q, span: #2);
  1, 1, 134, 1, 1;
  , hbrace(p, span: #3), hbrace(q, span: #2)) $
```



The braces cover the rows below them, so those rows start directly with their first cell.

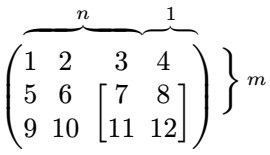
Braces over and under the cells

```
$ nicemat(
  1, 2, 3, 4, 5, 6;
  11, 12, 13, 14, 15, 16,
  hbrace((0, 0), (1, 2), A),
  hbrace((0, 3), (1, 5), B),
  hbrace((0, 0), (1, 5), "all",
    side: #bottom),
  vbrace((0, 0), (1, 5), 2 "rows")) $
```



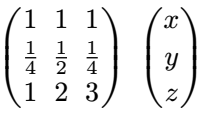
Braces over whole columns and rows

```
$ nicemat(
  1, 2, 3, 4; 5, 6, 7, 8; 9, 10, 11, 12,
  hbrace(n, cols: (0, 2)),
  hbrace(1, col: 3),
  vbrace(m, rows: (0, 1)),
  submatrix(rows: (1, 2), cols: (2, 3),
    delim: "[")) $
```



Sub-matrices

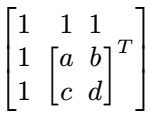
```
$ nicearray(
  1, 1, 1, x;
  frac(1, 4), frac(1, 2), frac(1, 4), y;
  1, 2, 3, z;
  submatrix((0, 0), (2, 2)),
  submatrix((0, 3), (2, 3))) $
```



Room is made for the delimiters automatically (`reserve: #false` to opt out).

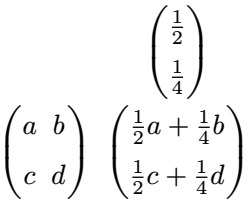
A transposed block

```
$ nicemat(delim: "[",
  1, 1, 1; 1, a, b; 1, c, d,
  submatrix((1, 1), (2, 2), delim: "[",
    sup: T)) $
```



A product

```
$ nicearray(cell-space: #2pt,
  , , frac(1, 2);
  , , frac(1, 4);
  a, b, frac(1, 2) a + frac(1, 4) b;
  c, d, frac(1, 2) c + frac(1, 4) d;
  submatrix((0, 2), (1, 2), slim: #true),
  submatrix((2, 0), (3, 1)),
  submatrix((2, 2), (3, 2))) $
```



9 Stacks of matrices

The matrices inside `nicemat-group[..]` share the widths of their columns and delimiters, so that their columns line up (nicematrix's `{NiceMatrixBlock}` with `auto-columns-width`).

Aligned columns

```
#nicemat-group[
  #let rm = nicemat.with(align: right,
    last-col: true, vlines: 4,
    map-cells: (i, j, it) => if j == 5 {
      text(blue, $script(it)$) } else { it })
  $ rm(12, -8, 7, 5, 3, ;
    3, -18, 12, 1, 4, ;
    -3, -46, 29, -2, -15, ;
    9, 10, -5, 4, 7, ) $
  $ rm(12, -8, 7, 5, 3, ;
    0, 64, -41, 1, 19, L_2 <- L_1 - 4 L_2;
    0, -192, 123, -3, -57, L_3 <- L_1 + 4 L_3;
    0, -64, 41, -1, -19, L_4 <- 3 L_1 - 4 L_4) $
  $ rm(12, -8, 7, 5, 3, ;
    0, 64, -41, 1, 19, ) $
]
```

$$\left(\begin{array}{rrrr|r} 12 & -8 & 7 & 5 & 3 \\ 3 & -18 & 12 & 1 & 4 \\ -3 & -46 & 29 & -2 & -15 \\ 9 & 10 & -5 & 4 & 7 \end{array}\right)$$

$$\left(\begin{array}{rrrr|r} 12 & -8 & 7 & 5 & 3 \\ 0 & 64 & -41 & 1 & 19 \\ 0 & -192 & 123 & -3 & -57 \\ 0 & -64 & 41 & -1 & -19 \end{array}\right) \begin{array}{l} L_2 \leftarrow L_1 - 4L_2 \\ L_3 \leftarrow L_1 + 4L_3 \\ L_4 \leftarrow 3L_1 - 4L_4 \end{array}$$

$$\left(\begin{array}{rrrr|r} 12 & -8 & 7 & 5 & 3 \\ 0 & 64 & -41 & 1 & 19 \end{array}\right)$$

10 Inline equations

Typst sets the cells of a `mat` smaller in an inline equation, and those of a `mat` nested in another one smaller again. A function cannot see whether it is called in an inline equation, so `nicemat` needs one show rule for that, at the top of the document:

`nicemat-setup`

```
#show: nicemat-setup
Inline, $nicemat(1, dots.c, 1; dots.v, dots.down, dots.v; 1, dots.c, 1)$
is as small as $mat(1, dots.c, 1; dots.v, dots.down, dots.v; 1, dots.c, 1)$,
nested ones shrink as with `mat`:
$ nicemat(nicemat(1, 2; 3, 4), 0; 0, I)
  quad mat(mat(1, 2; 3, 4), 0; 0, I) $
```

Inline, $\begin{pmatrix} 1 & \cdots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \cdots & 1 \end{pmatrix}$ is as small as $\begin{pmatrix} 1 & \cdots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \cdots & 1 \end{pmatrix}$, nested ones shrink as with `mat`:

$$\left(\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \quad 0 \\ 0 \quad I\right) \qquad \left(\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \quad 0 \\ 0 \quad I\right)$$

Without it, a `nicemat` always has the cells of a display equation. `small: #false` forces them; `small: #true` is the `small` of nicematrix, which also tightens the gaps and the dots.

11 Every `mat` at once, and defaults

A show rule

```
#show math.mat: nicemat
$ mat(1, dots.c, 1;
  dots.v, dots.down, dots.v;
  1, dots.c, 1) $
```

$$\begin{pmatrix} 1 & \cdots & 1 \\ \vdots & \ddots & \vdots \\ \vdots & \ddots & \vdots \\ 1 & \cdots & 1 \end{pmatrix}$$

The show rule keeps the geometry of `mat` and turns dots into leaders (nicematrix's `renew-matrix` and `renew-dots`); `#show: nicemat-setup.with(mat: true)` does the same together with the rule for inline equations. `#show math.mat: nicemat.with(..)` or `#let nicemat = nicemat.with(..)` set defaults for a whole document.

Markers in a plain `mat`

```
$ mat(1, cdots(), 1;
  vdots(), ddots(), vdots();
  1, cdots(), 1)
  quad
  mat(diagbox(x, y), cell(a, colspan: #2)) $
```

$$\begin{pmatrix} 1 & \cdots & 1 \\ \vdots & \ddots & \vdots \\ 1 & \cdots & 1 \end{pmatrix} \quad \begin{matrix} (x/y & a) \end{matrix}$$

Outside a `nicemat`, the markers fall back to their plain look.

12 Drawing with the geometry

`background` and `foreground` take content or a function receiving the geometry of the matrix (cells with their tiles and ink boxes, rows, columns, leaders), in coordinates ready for `place`. They replace the TikZ nodes, the `\CodeBefore` and the `\CodeAfter` of nicematrix.

Drawing on top

```
$ nicemat(foreground: #(g => {
    let (a, b) = (g.cells.at(0).at(0).ink,
                  g.cells.at(2).at(2).ink)
    place(line(
      start: (a.x + a.width, a.y + a.height),
      end: (b.x, b.y),
      stroke: (paint: red, dash: "dashed")))
  }),
  1, 0, 0; 0, 1, 0; 0, 0, 1) $
```

$$\begin{pmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{pmatrix}$$

A staircase in the background

```
$ nicearray(background: #(g => {
    let y(k) = g.rows.at(k).tile-top
    let x(k) = g.cols.at(k).tile-left
    place(polygon(fill: red.lighten(85%),
      stroke: 0.5pt,
      (x(3), y(6)), (x(3), y(7)), (x(4), y(7)),
      (x(4), y(8)), (x(5), y(8)), (x(5), y(6))))
  }),
  1; 1, 1; 1, 2, 1; 1, 3, 3, 1; 1, 4, 6, 4, 1;
  1, 5, 10, 10, 5, 1; 1, 6, 15, 20, 15, 6, 1;
  1, 7, 21, 35, 35, 21, 7, 1;
  1, 8, 28, 56, 70, 56, 28, 8, 1) $
```

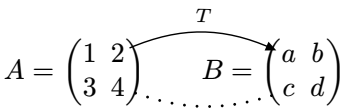
$$\begin{array}{ccccccccc} 1 & & & & & & & & \\ 1 & 1 & & & & & & & \\ 1 & 2 & 1 & & & & & & \\ 1 & 3 & 3 & 1 & & & & & \\ 1 & 4 & 6 & 4 & 1 & & & & \\ 1 & 5 & 10 & 10 & 5 & 1 & & & \\ 1 & 6 & 15 & 20 & 15 & 6 & 1 & & \\ 1 & 7 & 21 & 35 & 35 & 21 & 7 & 1 & \\ 1 & 8 & 28 & 56 & 70 & 56 & 28 & 8 & 1 \end{array}$$

13 Named matrices

A matrix with a `name` can be reached from outside: `nicemat-cell(name, (i, j))` gives the position of a cell on the page (in a `context`, `none` until the matrix is laid out), and `nicemat-connect((name, (i, j)), (name, (i, j)))` draws between cells of different matrices of a page, with the options of `connect` (TikZ's `remember picture`). Put it anywhere on that page, for instance just after the equation: it takes no room.

Between two matrices

```
$ A = nicemat(name: "A", 1, 2; 3, 4)
  wide
  B = nicemat(name: "B", a, b; c, d) $
#nicemat-connect(("A", (0, 1)), ("B", (0, 0)),
  bend: 25deg, above: $T$)
#nicemat-connect(("A", (1, 1)), ("B", (1, 0)),
  bend: -20deg, stroke: auto, marks: none)
```



With `stroke: auto` the line is dotted, like a leader.

14 Showcase

These examples of pavemat mix paths, flood fills and single cells.

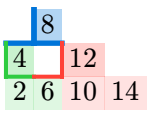
A network topology

```
$ nicemat(delim: "|",
  "R1", "SW1", "SW2", "FW", "WAN";
  "PC1", "PC2", "SRV1", "SRV2", "ISP";
  "AP1", "AP2", "DB", "LB", "CDN";
  "IOT1", "IOT2", "STOR", "BAK", "MON";
  "DEV1", "DEV2", "TEST", "PROD", "LOG",
  // Main data path (red backbone)
  pave("DSDSDSDS", stroke: #(red + 3pt)),
  // Backup path (blue failover)
  pave("ADADADAD", from: (0, 4), stroke: #(
    paint: blue, thickness: 2pt, dash: "dashed")),
  // Network perimeter (green boundary)
  pave("DDDDSSSSAAAAWWWW", stroke: #(
    paint: green, thickness: 1pt, dash: "dotted")),
  fill: #gray.transparentize(98%),
  region((2, 2), (2, 2), fill: #orange.transparentize(70%)),
  region((0, 3), (0, 3), fill: #purple.transparentize(80%)),
  region((1, 4), (1, 4), fill: #blue.transparentize(85%))) $
```



A binary search tree

```
#let one(i, j, c) = region((i, j), (i, j), fill: c)
$ nicearray(
  "", 8, "", "";
  4, "", 12, "";
  2, 6, 10, 14,
  // Root to the subtrees
  pave("SA", from: (0, 1), stroke: #(blue + 2pt)),
  pave("SD", from: (0, 1), stroke: #(blue + 2pt)),
  // Left and right subtrees
  pave("S", from: (1, 0), stroke: #(green + 1.5pt)),
  pave("SD", from: (1, 0), stroke: #(green + 1.5pt)),
  pave("SA", from: (1, 2), stroke: #(red + 1.5pt)),
  pave("S", from: (1, 2), stroke: #(red + 1.5pt)),
  one(0, 1, #blue.transparentize(70%)),
  one(1, 0, #green.transparentize(75%)),
  one(1, 2, #red.transparentize(75%)),
  one(2, 0, #green.transparentize(85%)),
  one(2, 1, #green.transparentize(85%)),
  one(2, 2, #red.transparentize(85%)),
  one(2, 3, #red.transparentize(85%))) $
```



Gaussian elimination, annotated

```
#let one(i, j, c) = region((i, j), (i, j), fill: c)
$ nicemat(delim: "[",
  2, -1, 3, 7;
  1, 2, -1, 4;
  3, 1, 2, 10,
  // Separator before the last column
  pave("SSS", from: (0, 3), stroke: #(black + 0.5pt)),
  flood((0, 0), fill: #yellow.transparentize(90%)),
  one(0, 0, #red.transparentize(80%)),
  one(0, 3, #blue.transparentize(80%)),
  one(1, 3, #blue.transparentize(80%)),
  one(2, 3, #blue.transparentize(80%))) $
```

$$\left[\begin{array}{ccc|c} 2 & -1 & 3 & 7 \\ 1 & 2 & -1 & 4 \\ 3 & 1 & 2 & 10 \end{array}\right]$$

A path-finding trace

```
#let one(i, j, c) = region((i, j), (i, j), fill: c)
#let wall = gray.transparentize(40%)
$ nicemat(stroke: #(dash: "dashed", thickness: 0.5pt),
  fill: #blue.transparentize(95%),
  S, ·, ×, ·, ·;
  ·, ×, ×, ·, ×;
  ·, ·, ·, ·, G;
  ×, ×, ·, ×, ·;
  ·, ·, ·, ·, ·,
  // Optimal path with growing confidence
  pave("D(thickness: 1pt, paint: orange)D(thickness: 2pt, paint: orange)S(thickness: 3pt, paint:
green)S(thickness: 3pt, paint: green)"),
  // Explored nodes (backtracking)
  pave("DSA", from: (1, 1), stroke: #(paint: gray, thickness: 1pt, dash: "dashed")),
  pave("AS", from: (2, 1), stroke: #(paint: gray, thickness: 1pt, dash: "dashed")),
  one(0, 2, #wall), one(1, 1, #wall), one(1, 2, #wall),
  one(1, 4, #wall), one(3, 0, #wall), one(3, 1, #wall),
  one(3, 3, #wall),
  one(0, 0, #green.transparentize(60%)),
  one(2, 4, #red.transparentize(60%))) $
```

$$\begin{pmatrix} S & \cdot & \times & \cdot & \cdot \\ \cdot & \times & \times & \cdot & \times \\ \cdot & \cdot & \cdot & \cdot & G \\ \times & \times & \cdot & \times & \cdot \\ \cdot & \cdot & \cdot & \cdot & \cdot \end{pmatrix}$$

III. Reference

1 `nicemat` and `nicearray`

`nicemat(..rows, ..options)` takes the rows (from the math syntax, arrays in code, a flat list for a single row, or a `mat` element). `nicearray` is `nicemat` with `delim: none`.

option	default	meaning (nicematrix / pavemat)
`delim`	from `mat`	`"("`, `"["`, `"{"`, `"|"`, `"||"`, any extensible symbol (`"↓"`…), a pair (`("[", ")")`) or `none`
`delim-fill`	`auto`	color of the delimiters (`delimiters/color`)
`align`	from `mat`	alignment, array (one per column, the last one repeated) or `(i, j) => alignment` (`l` / `c` / `r`, `columns-type`)
`gap`, `row-gap`, `column-gap`	from `mat`	a length, or an array with one value per gap (`\\[..]`, `@{..}`)
`augment`	`none`	as in `mat`
`hlines`, `vlines`	`none`	`true`, an index or a list of indices, counted within the matrix (`hlines`, `vlines`)
`hvlines`	`false`	`true`: `hlines` and `vlines` both `true` (`hvlines`)
`borders`	`auto`	outer rules of `hlines: true` / `vlines: true`: `auto` only without delimiters, `true`, `false` (`hvlines-except-borders`)
`stroke`	`0.05em`	stroke of the rules and paths (`rules/color`, `rules/width`; the `stroke` of pavemat)
`fill`	`none`	color, array (cycled over the columns) or `(i, j) => color` (`\rowcolors`, `\chessboardcolors`…; the global fill of pavemat)
`corners`	`none`	`true` or alignments: `top + left`… and `top`, `right`… (`corners`)
`first-row` … `last-col`	`false`	exterior rows and columns
`map-cells`	`none`	`(i, j, body) => body` (`code-for-first-row`, `\RowStyle`)
`cell-space`	`0pt`	minimum room between the ink and the edges of the rows; length or `(top:, bottom:)` (`cell-space-limits`)
`margin`	`auto`	room inside the delimiters; length or dictionary (`margin`, `extra-margin`)
`column-width`	`auto`	a minimum length, one per column (`(auto, 1cm)`, like `w{c}{1cm}`), or `"equal"` (`columns-width`)
`baseline`	`horizon`	`top`, `horizon`, `bottom`, a row, or `(line: k)` to put a rule on the axis (`baseline`)
`small`	`auto`	`auto`: the cells of `mat` (smaller in inline equations under `nicemat-setup`, and when nested); `true`: script-style cells, tighter gaps and dots (`small`); `false`: display cells
`dots`	`(:)`	options of all the leaders (`xdots/..`)
`background`, `foreground`	`none`	content or `(geometry) => content` (`\CodeBefore`, `\CodeAfter`, TikZ nodes)
`name`	`none`	a name for `nicemat-cell` and `nicemat-connect` (`name`, `remember picture`)
`debug`	`false`	`true`: indices and tiles of the cells, numbers of the rules, hidden path segments; `"cells"` or `"rules"` for one of them (`\ShowCellNames`; `debug`)

2 Leaders

`ldots`, `cdots`, `vdots`, `ddots`, `iddots` and the symbols `dots.h`, `dots.c`, `dots.v`, `dots.down`, `dots.up`. The same options are accepted by `nicemat(dots: (..))` for all the leaders, and by `dotline(from, to)` and `connect(from, to)`, which also take `bend`.

option	default	meaning (nicematrix / pavemat)
`symbols`	`true`	(in `dots` only) turn the dots symbols into leaders (`renew-dots`)
`span`	`1`	(one leader) cover several columns or rows (`\Hdotsfor`, `\Vdotsfor`)
`above`, `below`, `middle`	`none`	(one leader) labels (`^`, `_`, `:`)
`horizontal-labels`	`false`	keep the labels horizontal
`fill`	text color	color of the dots (`color`), or `(i, j) => color` of the cell where the leader starts (`none`: text color)
`stroke`	`auto`	`auto` for round dots, or a stroke for a line (`line-style`)
`radius`, `spacing`	`0.53pt`, `0.45em`	size and distance of the dots (`radius`, `inter`); 70% of that in small cells
`shorten`, `-start`, `-end`	`0.3em`	room at the closed ends; 70% in small cells
`bend`	`0`	(`dotline`, `connect`) an angle, or a number of degrees: curve to the left (`bend left`), or to the right when negative
`nullify`	`false`	the leaders take no room (`nullify`)
`parallel`	`true`	(in `dots` only) parallel diagonals (`parallelize-diags`)
`first`	`false`	(one leader) the reference diagonal (`draw-first`)
`marks`, `mark-size`	`none`, `4pt`	arrow tips `"->"`, `"<-"`, `"<->"`
`label-fill`	page fill	background of the `middle` labels

3 Markers

Markers are written among the cells. Coordinates are `(row, col)` as written; paths use grid points counted within the matrix.

marker	options
`cell(body)`	`rowspan`, `colspan`, `fill`, `stroke` (`true`, a stroke, or one per side), `radius`, `outset` (length or `(x:, y:)`), `align`, `transparent` (rules cross it), `empty` (`\NotEmpty`)
`diagbox(lower, upper)`	a cell slashed diagonally (`\diagbox`), also as the body of a spanning `cell`; `stroke`
`hline()`, `vline()`	`start`, `end` (excluded), `stroke`, `y` / `x` (`\Hline`, `|`, `custom-line`)
`hbrace(label, span:)`, `vbrace(label, span:)`	in exterior rows and columns (`\Hbrace`, `\Vbrace`); `fill`, `shift`
`hbrace(from, to, label)`, `vbrace(from, to, label)`	`side`, `shorten`, `shift`, `fill` (`\OverBrace`, `\UnderBrace`); `hbrace(label, cols: (a, b))` and `vbrace(label, rows: (a, b))` for whole columns or rows
`submatrix(from, to)`	or `row:`, `rows:`, `col:`, `cols:`; `delim`, `sup`, `sub`, `hlines`, `vlines` (in the submatrix), `stroke`, `slim`, `xshift`, `left-xshift`, `right-xshift`, `extra-height`, `fill`, `reserve`, `bound` (`\SubMatrix`)

marker	options
`region(from, to)`	or `row:`, `rows:`, `col:`, `cols:`; `fill`, `stroke` (one per side allowed), `radius`, `outset`, `fit` (`"cells"`, `"content"`), `above` (`\rectanglecolor`; single-cell fills of pavemat)
`dotline(from, to)`	the options of the leaders and `bend` (`\line`)
`connect(from, to)`	a `dotline` with a solid line and an arrow head (`marks: "->"`); same options
`pave(path)`	`from` (grid point or corner), `stroke`, `chars` (the paths of pavemat)
`flood(from)`	`fill` (the region fills of pavemat)

4 Functions

function	use
`#show: nicemat-setup`	matrices of inline equations and nested ones sized like `mat`; `.with(mat: true)` also turns every `mat` into a `nicemat`
`nicemat-group[..]`	the matrices inside share their column widths (`{NiceMatrixBlock}`, `auto-columns-width`)
`nicemat-cell(name, (i, j))`	in a `context`: `none`, or the cell of a named matrix on the page: `page`, `x`, `y`, `width`, `height` (the tile), `ink` (the content), `baseline`
`nicemat-connect((name, (i, j)), (name, (i, j)))`	a line between cells of named matrices on the same page; the options of `connect`

5 Geometry

The function given to `background` / `foreground` receives a dictionary with `width`, `height`, `baseline`, `axis`, `main` (the rectangle of the matrix inside its delimiters), `rows` (`top`, `baseline`, `bottom`, `tile-top`, `tile-bottom`), `cols` (`left`, `right`, `tile-left`, `tile-right`), `cells.at(i).at(j)` (the tile `x`, `y`, `width`, `height`, plus `ink`, `baseline`, `empty`, `kind`, `origin`) and `leaders` (`dir`, `from`, `to`, `open`, `start`, `end`). All the coordinates are relative to the top-left corner of the matrix.

6 Differences with nicematrix and pavemat

- The cells are set in text style, as in LaTeX and as `mat` in a display equation. In inline equations and nested matrices they follow `mat` (script style, smaller again when nested) once `#show: nicemat-setup` is there; Typst gives no other way to detect an inline equation.
- Typst's matrices are tighter than LaTeX's (0.5em between the columns instead of 10pt): the leaders over empty columns are a little shorter.
- Nothing needs a second compilation (except `nicemat-group`, which converges on its own).
- Sub-matrices and braces make room for themselves instead of overlapping their surroundings; the braces come from the math font rather than TikZ.
- The paths of pavemat use the stroke of the matrix: a stroke given to one path is added to it (in pavemat it replaces it), and the default is a solid rule rather than a dashed line. As in pavemat, a matrix with paths or flood fills gets a margin of half a gap around its cells (unless `margin` is given).
- The fills of pavemat are split between the `fill` option (global), `region` (a cell or a rectangle) and `flood` (a region bounded by paths); they are drawn in the order of the arguments.

IV. The nicematrix manual, example by example

This part takes, in order, the examples of the manual of nicematrix (version 7.11) that concern the mathematical environments, and gives for each one the original code (with the page of the manual where it appears), the Typst code, and the result produced by that Typst code. The examples with `{NiceTabular}` and those relying on TikZ decorations are listed at the end.

The LaTeX code is taken from the manual of nicematrix (© François Pantigny), distributed under the LaTeX Project Public License 1.3c; only the `\emph{..}` highlighting of the manual was removed.

1 The vertical space between the rows

LaTeX — nicematrix manual, p. 4

```
$\begin{pmatrix}
\frac{1}{2} & -\frac{1}{2} \\
\frac{1}{3} & \frac{1}{4} \\
\end{pmatrix}$
```

Typst

```
$ mat(frac(1, 2), -frac(1, 2);
      frac(1, 3), frac(1, 4)) $
```

$$\begin{pmatrix} \frac{1}{2} & -\frac{1}{2} \\ \frac{1}{3} & \frac{1}{4} \end{pmatrix}$$

Typst's own `mat`, for comparison.

LaTeX — nicematrix manual, p. 5

```
$\begin{pNiceMatrix}[cell-space-limits = 1pt]
\frac12 & -\frac12 \\
\frac13 & \frac14 \\
\end{pNiceMatrix}$
```

Typst

```
$ nicemat(cell-space: #1pt,
  frac(1, 2), -frac(1, 2);
  frac(1, 3), frac(1, 4)) $
```

$$\begin{pmatrix} \frac{1}{2} & -\frac{1}{2} \\ \frac{1}{3} & \frac{1}{4} \end{pmatrix}$$

2 The vertical position of the arrays

LaTeX — nicematrix manual, p. 5

```
$A = \begin{pNiceMatrix}[baseline=2]
\frac{1}{\sqrt{1+p^2}} & p & 1-p \\
1 & 1 & 1 \\
1 & p & 1+p
\end{pNiceMatrix}$
```

Typst

```
$ A = nicemat(baseline: 1,
  frac(1, sqrt(1 + p^2)), p, 1 - p;
  1, 1, 1;
  1, p, 1 + p) $
```

$$A = \begin{pmatrix} \frac{1}{\sqrt{1+p^2}} & p & 1-p \\ 1 & 1 & 1 \\ 1 & p & 1+p \end{pmatrix}$$

Rows are numbered from 0: `baseline=2` becomes `baseline: 1`.

LaTeX — nicematrix manual, p. 5

```
\begin{enumerate}
\item an item
\smallskip
\item \renewcommand{\arraystretch}{1.2}
$\begin{NiceArray}[t]{lcccccc}
\hline
n   & 0 & 1 & 2 & 3 & 4  & 5 \\
u_n & 1 & 2 & 4 & 8 & 16 & 32
\hline
\end{NiceArray}$
\end{enumerate}
```

Typst

```
+ an item
+ $nicearray(baseline: #top,
    align: #(left, center),
    row-gap: #0.45em,
    margin: #(x: 0.3em, y: 0.25em),
    hline();
    n, 0, 1, 2, 3, 4, 5;
    u_n, 1, 2, 4, 8, 16, 32;
    hline())$
```

1. an item
2. n 0 1 2 3 4 5
   u_n 1 2 4 8 16 32

`align` takes one alignment per column; the last one is repeated.

LaTeX — nicematrix manual, p. 6

```
\begin{enumerate}
\item an item
\smallskip
\item
$\begin{NiceArray}[t]{lccccc}
\toprule
n   & 0 & 1 & 2 & 3 & 4  & 5 \\
\midrule
u_n & 1 & 2 & 4 & 8 & 16 & 32
\bottomrule
\end{NiceArray}$
\end{enumerate}
```

Typst

```
+ an item
+ $nicearray(baseline: #top,
    align: #(left, center),
    row-gap: #0.6em,
    margin: #(x: 0.3em, y: 0.35em),
    hline(stroke: #0.08em);
    n, 0, 1, 2, 3, 4, 5;
    hline(stroke: #0.05em);
    u_n, 1, 2, 4, 8, 16, 32;
    hline(stroke: #0.08em))$
```

1. an item
2. n 0 1 2 3 4 5
   u_n 1 2 4 8 16 32

The rules of booktabs are simply rules of different thicknesses.

LaTeX — nicematrix manual, p. 6

```
\NiceMatrixOptions{cell-space-limits=1pt}
$A=\begin{pNiceArray}{cc|cc}[baseline=line-3]
\dfrac{1}{A} & \dfrac{1}{B} & 0 & 0 \\
\dfrac{1}{C} & \dfrac{1}{D} & 0 & 0 \\
\hline
0 & 0 & A & B \\
0 & 0 & D & D \\
\end{pNiceArray}$
```

Typst

```
$ A = nicemat(baseline: #(line: 2),
  cell-space: #1pt, vlines: 2,
  display(1/A), display(1/B), 0, 0;
  display(1/C), display(1/D), 0, 0;
  hline();
  0, 0, A, B;
  0, 0, D, D) $
```

$$A = \left(\begin{array}{cc|cc} \frac{1}{A} & \frac{1}{B} & 0 & 0 \\ \frac{1}{C} & \frac{1}{D} & 0 & 0 \\ \hline 0 & 0 & A & B \\ 0 & 0 & D & D \end{array}\right)$$

`baseline: (line: 2)` puts the rule number 2 (counted from 0 within the matrix, like `hlines`) on the math axis.

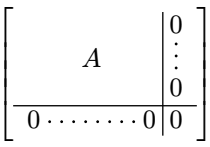
3 The blocks

LaTeX — nicematrix manual, p. 7

```
$\begin{bNiceArray}{cw{c}{1cm}c|c}[margin]
  \Block{3-3}{A} & & & 0 \\
  & & & \Vdots \\
  & & & 0 \\
  \hline
  0 & \Cdots& 0 & 0
\end{bNiceArray}$
```

Typst

```
$ nicemat(delim: "[", margin: #(x: 5pt),
  column-width: #(auto, 1cm), vlines: 3,
  cell(A, rowspan: #3, colspan: #3), 0;
  dots.v;
  0;
  hline();
  0, dots.c, 0, 0) $
```



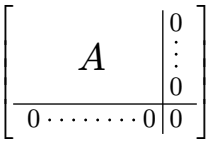
`w{c}{1cm}` becomes a minimum width for that column. The positions covered by the block are skipped: the next rows only give their last cell.

LaTeX — nicematrix manual, p. 7

```
$\begin{bNiceArray}{cw{c}{1cm}c|c}[margin]
  \Block{3-3}<\Large>{A} & & & 0 \\
  0 & & & \Vdots \\
  & & & 0 \\
  \hline
  0 & \Cdots& 0 & 0
\end{bNiceArray}$
```

Typst

```
$ nicemat(delim: "[", margin: #(x: 5pt),
  column-width: #(auto, 1cm), vlines: 3,
  cell(#text(1.44em)[$A$],
    rowspan: #3, colspan: #3), 0;
  dots.v;
  0;
  hline();
  0, dots.c, 0, 0) $
```



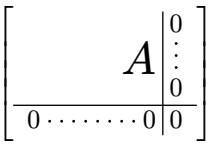
`<\Large>` becomes a `text` size. The stray `0` written under the block in the LaTeX code has no equivalent: a covered position can't hold a cell.

LaTeX — nicematrix manual, p. 10

```
$\begin{bNiceArray}{cw{c}{1cm}c|c}[margin]
  \Block[r]{3-3}<\LARGE>{A} & & & 0 \\
  & & & \Vdots \\
  & & & 0 \\
  \hline
  0 & \Cdots& 0 & 0
\end{bNiceArray}$
```

Typst

```
$ nicemat(delim: "[", margin: #(x: 5pt),
  column-width: #(auto, 1cm), vlines: 3,
  cell(#text(1.728em)[$A$], align: #right,
    rowspan: #3, colspan: #3), 0;
  dots.v;
  0;
  hline();
  0, dots.c, 0, 0) $
```



4 The rules

LaTeX — nicematrix manual, p. 14

```
$\begin{NiceArray}{c|ccc} \toprule
  a & b & c & d \\ \midrule
  1 & 2 & 3 & 4 \\
  1 & 2 & 3 & 4 \\ \bottomrule
\end{NiceArray}$
```

Typst

```
$ nicearray(vlines: 1,
  margin: #(x: 0.3em, y: 0.3em),
  hline(stroke: #0.08em);
  a, b, c, d;
  hline(stroke: #0.05em);
  1, 2, 3, 4;
  1, 2, 3, 4;
  hline(stroke: #0.08em)) $
```

a	b	c	d
1	2	3	4
1	2	3	4

LaTeX — nicematrix manual, p. 16

```
$\begin{pNiceMatrix}[vlines,rules/width=0.2pt]
  1 & 2 & 3 & 4 & 5 & 6 \\
  1 & 2 & 3 & 4 & 5 & 6 \\
  1 & 2 & 3 & 4 & 5 & 6
\end{pNiceMatrix}$
```

Typst

```
$ nicemat(vlines: #true, stroke: #0.2pt,
  1, 2, 3, 4, 5, 6;
  1, 2, 3, 4, 5, 6;
  1, 2, 3, 4, 5, 6) $
```

$$\begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 \\ 1 & 2 & 3 & 4 & 5 & 6 \\ 1 & 2 & 3 & 4 & 5 & 6 \end{pmatrix}$$

With delimiters, `vlines: #true` doesn't draw the outer rules.

LaTeX — nicematrix manual, p. 17

```
$\begin{NiceArray}{c||ccccc}[hlines,vlines]
    & a & b & c & d & e \\ \Hline\Hline
  x & 0 & 0 & 0 & 0 & 0 \\
  y & 0 & 0 & 0 & 0 & 0 \\
  z & 0 & 0 & 0 & 0 & 0 \\
\end{NiceArray}$
```

Typst

```
$ nicearray(hlines: #true, vlines: #true,
  , vline(), vline(), a, b, c, d, e;
  hline(); hline();
  x, 0, 0, 0, 0, 0;
  y, 0, 0, 0, 0, 0;
  z, 0, 0, 0, 0, 0) $
```

	a	b	c	d	e
x	0	0	0	0	0
y	0	0	0	0	0
z	0	0	0	0	0

Two markers at the same place give a double rule, even when `hlines` / `vlines` are in force.

LaTeX — nicematrix manual, p. 18

```
$\begin{NiceArray}{*{5}{c}}[hvlines]
  \diagbox{x}{y}  & e & a & b & c \\
  e & e & a & b & c \\
  a & a & e & c & b \\
  b & b & c & e & a \\
  c & c & b & a & e
\end{NiceArray}$
```

Typst

```
$ nicearray(hlines: #true, vlines: #true,
  diagbox(x, y), e, a, b, c;
  e, e, a, b, c;
  a, a, e, c, b;
  b, b, c, e, a;
  c, c, b, a, e) $
```

x \ y	e	a	b	c
e	e	a	b	c
a	a	e	c	b
b	b	c	e	a
c	c	b	a	e

LaTeX — nicematrix manual, p. 19

```
$\begin{NiceArray}{*{5}{c}}[hvlines]
  \Block{2-2}{\diagbox{x}{y}} &  & a & b & c \\
    &   & a & b & c \\
  a & a & e & c & b \\
  b & b & c & e & a \\
  c & c & b & a & e
\end{NiceArray}$
```

Typst

```
$ nicearray(hlines: #true, vlines: #true,
  cell(diagbox(x, y),
    rowspan: #2, colspan: #2), a, b, c;
  a, b, c;
  a, a, e, c, b;
  b, b, c, e, a;
  c, c, b, a, e) $
```

x \ y		a	b	c
		a	b	c
a	a	e	c	b
b	b	c	e	a
c	c	b	a	e

LaTeX — nicematrix manual, p. 19

```
$\begin{NiceArray}{*{5}{c}}[hvlines]
  \Block{2-2}{\diagbox{}{}} & y & a & b & c \\
  x &   & a & b & c \\
  a & a & e & c & b \\
  b & b & c & e & a \\
  c & c & b & a & e
\end{NiceArray}$
```

Typst

```
$ nicearray(hlines: #true, vlines: #true,
  cell(diagbox(x, y),
    rowspan: #2, colspan: #2), a, b, c;
  a, b, c;
  a, a, e, c, b;
  b, b, c, e, a;
  c, c, b, a, e) $
```

x \ y		a	b	c
		a	b	c
a	a	e	c	b
b	b	c	e	a
c	c	b	a	e

In LaTeX, `x` and `y` are written in cells covered by the block. Covered positions can't hold cells here, so the labels are given to `diagbox`, as in the previous example.

LaTeX — nicematrix manual, p. 21

```
$\begin{pNiceMatrix}
1 & 2 & 3 & 4 & 5 \\
\hdashedline
6 & 7 & 8 & 9 & 10 \\
\cdashedline{1,4-5}
11 & 12 & 13 & 14 & 15
\end{pNiceMatrix}$
```

Typst

```
$ nicemat(
  1, 2, 3, 4, 5;
  hline(stroke: #(dash: "dashed"));
  6, 7, 8, 9, 10;
  hline(end: #1, stroke: #(dash: "dashed")),
    hline(start: #3, stroke: #(dash: "dashed"));
  11, 12, 13, 14, 15) $
```

$$\begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ \hdashline 6 & 7 & 8 & 9 & 10 \\ \cdashline{1-2}\cdashline{4-5} 11 & 12 & 13 & 14 & 15 \end{pmatrix}$$

`\cdashedline{1,4-5}` : two partial rules; `start` and `end` are columns (from 0, `end` excluded).

LaTeX — nicematrix manual, p. 21

```
$\begin{pNiceArray}{cccc;c}
1 & 2 & 3 & 4 & 5 \\
6 & 7 & 8 & 9 & 10 \\
11 & 12 & 13 & 14 & 15
\end{pNiceArray}$
```

Typst

```
$ nicemat(
  1, 2, 3, 4, vline(stroke: #(dash: "dashed")), 5;
  6, 7, 8, 9, 10;
  11, 12, 13, 14, 15) $
```

$$\left(\begin{array}{cccc:c} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \end{array}\right)$$

A `vline()` written in a row is drawn after the cell it follows.

LaTeX — nicematrix manual, p. 21

```
$\begin{pNiceMatrix}
1 & 2 & 3 & 4 & 5 \\
\myhdashedline
6 & 7 & 8 & 9 & 10 \\
\mycdashedline{1,4-5}
11 & 12 & 13 & 14 & 15
\end{pNiceMatrix}$
```

Typst

```
#let myhdashedline(..args) = hline(
  stroke: (dash: "dashed"), ..args)
$ nicemat(row-gap: #(10pt, 10pt),
  1, 2, 3, 4, 5;
  myhdashedline();
  6, 7, 8, 9, 10;
  myhdashedline(end: #1),
    myhdashedline(start: #3);
  11, 12, 13, 14, 15) $
```

$$\left(\begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \\ \hdashline 6 & 7 & 8 & 9 & 10 \\ \hdashline 11 & 12 & 13 & 14 & 15 \end{array}\right)$$

`custom-line` (p. 22) becomes a function. Rules never take room, so `total-width = 10pt` becomes a larger `row-gap`.

LaTeX — nicematrix manual, p. 22

```
$\begin{pNiceMatrix}
1 & 2 & 3 & 4 & 5 \\
\hdottedline
6 & 7 & 8 & 9 & 10 \\
\cdottedline{1,4-5}
11 & 12 & 13 & 14 & 15
\end{pNiceMatrix}$
```

Typst

```
#let dotted = (thickness: 1pt,
  dash: (0pt, 3.5pt), cap: "round")
$ nicemat(
  1, 2, 3, 4, 5;
  hline(stroke: #dotted);
  6, 7, 8, 9, 10;
  hline(end: #1, stroke: #dotted),
    hline(start: #3, stroke: #dotted);
  11, 12, 13, 14, 15) $
```

$$\left(\begin{array}{ccccc} 1 & 2 & 3 & 4 & 5 \\ \hdashline 6 & 7 & 8 & 9 & 10 \\ \hdashline 11 & 12 & 13 & 14 & 15 \end{array}\right)$$

Round dots are a dash pattern of zero-length dashes with round caps.

LaTeX — nicematrix manual, p. 22

```
$\begin{pNiceArray}{cccc:c}
1 & 2 & 3 & 4 & 5 \\
6 & 7 & 8 & 9 & 10 \\
11 & 12 & 13 & 14 & 15
\end{pNiceArray}$
```

Typst

```
#let dotted = (thickness: 1pt,
  dash: (0pt, 3.5pt), cap: "round")
$ nicemat(
  1, 2, 3, 4, vline(stroke: #dotted), 5;
  6, 7, 8, 9, 10;
  11, 12, 13, 14, 15) $
```

$$\left(\begin{array}{cccc:c} 1 & 2 & 3 & 4 & 5 \\ 6 & 7 & 8 & 9 & 10 \\ 11 & 12 & 13 & 14 & 15 \end{array}\right)$$

5 The color of the background of the rows and columns

LaTeX — nicematrix manual, p. 26

```
$\begin{pNiceMatrix}[r,margin]
\CodeBefore
  \chessboardcolors{red!15}{blue!15}
\Body
  1  & -1 & 1 \\
  -1 & 1  & -1 \\
  1  & -1 & 1
\end{pNiceMatrix}$
```

Typst

```
$ nicemat(align: #right, margin: #(x: 5pt),
  fill: #((i, j) => if calc.even(i + j) {
    red.lighten(85%) } else { blue.lighten(85%) }),
  1, -1, 1;
  -1, 1, -1;
  1, -1, 1) $
```

$$\begin{pmatrix} 1 & -1 & 1 \\ -1 & 1 & -1 \\ 1 & -1 & 1 \end{pmatrix}$$

`\chessboardcolors`, `\rowcolors`, `\columncolor`... are all functions `(i, j) => color`.

LaTeX — nicematrix manual, p. 27

```
$\begin{NiceArray}{lll}[hvlines]
\CodeBefore
  \rowcolor{red!15}{1,3-5,8-}
\Body
  a_1 & b_1 & c_1 \\
  a_2 & b_2 & c_2 \\
  a_3 & b_3 & c_3 \\
  a_4 & b_4 & c_4 \\
  a_5 & b_5 & c_5 \\
  a_6 & b_6 & c_6 \\
  a_7 & b_7 & c_7 \\
  a_8 & b_8 & c_8 \\
  a_9 & b_9 & c_9 \\
  a_{10} & b_{10} & c_{10} \\
\end{NiceArray}$
```

Typst

```
$ nicearray(align: #left,
  hlines: #true, vlines: #true,
  fill: #((i, j) => if i + 1 in (1, 3, 4, 5)
    or i + 1 >= 8 { red.lighten(85%) }),
  a_1, b_1, c_1;
  a_2, b_2, c_2;
  a_3, b_3, c_3;
  a_4, b_4, c_4;
  a_5, b_5, c_5;
  a_6, b_6, c_6;
  a_7, b_7, c_7;
  a_8, b_8, c_8;
  a_9, b_9, c_9;
  a_10, b_10, c_10) $
```

a_1	b_1	c_1
a_2	b_2	c_2
a_3	b_3	c_3
a_4	b_4	c_4
a_5	b_5	c_5
a_6	b_6	c_6
a_7	b_7	c_7
a_8	b_8	c_8
a_9	b_9	c_9
a_{10}	b_{10}	c_{10}

`\rowcolor{red!15}{1,3-5,8-}`, with the row numbers of the manual.

6 The width of the columns

LaTeX — nicematrix manual, p. 33

```
$\begin{pNiceMatrix}[columns-width = 1cm]
  1  & 12 & -123 \\
  12 & 0  & 0    \\
  4  & 1  & 2
\end{pNiceMatrix}$
```

Typst

```
$ nicemat(column-width: #1cm,
  1, 12, -123;
  12, 0, 0;
  4, 1, 2) $
```

$$\begin{pmatrix} 1 & 12 & -123 \\ 12 & 0 & 0 \\ 4 & 1 & 2 \end{pmatrix}$$

LaTeX — nicematrix manual, p. 33

```
$\begin{pNiceMatrix}[columns-width = auto]
  1  & 12 & -123 \\
  12 & 0  & 0    \\
  4  & 1  & 2
\end{pNiceMatrix}$
```

Typst

```
$ nicemat(column-width: "equal",
  1, 12, -123;
  12, 0, 0;
  4, 1, 2) $
```

$$\begin{pmatrix} 1 & 12 & -123 \\ 12 & 0 & 0 \\ 4 & 1 & 2 \end{pmatrix}$$

LaTeX — nicematrix manual, p. 34

```
\NiceMatrixOptions{columns-width=10mm}
$\begin{pNiceMatrix}
a & b \\ c & d
\end{pNiceMatrix}
=
\begin{pNiceMatrix}
1   & 1245 \\ 345 & 2
\end{pNiceMatrix}$
```

Typst

```
#let nm = nicemat.with(column-width: 10mm)
$ nm(a, b; c, d) = nm(1, 1245; 345, 2) $
```

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix} = \begin{pmatrix} 1 & 1245 \\ 345 & 2 \end{pmatrix}$$

`\NiceMatrixOptions` becomes `.with`. (A function used in math needs a name of two letters or more.)

LaTeX — nicematrix manual, p. 34

```
\begin{NiceMatrixBlock}[auto-columns-width]
$\begin{array}{c}
\begin{bNiceMatrix}
 9 & 17 \\ -2 & 5
 \end{bNiceMatrix} \\ \\
\begin{bNiceMatrix}
 1   & 1245345 \\  345 & 2
\end{bNiceMatrix} \\
\end{array}$
\end{NiceMatrixBlock}
```

Typst

```
#nicemat-group[
  $ vec(delim: #none, gap: #1em,
    nicemat(delim: "[", 9, 17; -2, 5),
    nicemat(delim: "[", 1, 1245345; 345, 2)) $
]
```

$$\begin{bmatrix} 9 & 17 \\ -2 & 5 \end{bmatrix}$$

$$\begin{bmatrix} 1 & 1245345 \\ 345 & 2 \end{bmatrix}$$

`{NiceMatrixBlock}[auto-columns-width]` becomes `nicemat-group`.

7 The exterior rows and columns

LaTeX — nicematrix manual, p. 36

```
$\begin{pNiceMatrix}[first-row,last-row,first-
col,last-col,xdots/nullify]
       & C_1    & \Cdots &        & C_4    &
\\
L_1    & a_{11} & a_{12} & a_{13} & a_{14} & L_1
\\
\Vdots & a_{21} & a_{22} & a_{23} & a_{24} & \Vdots
\\
       & a_{31} & a_{32} & a_{33} & a_{34} &
\\
L_4    & a_{41} & a_{42} & a_{43} & a_{44} & L_4
\\
       & C_1    & \Cdots &        & C_4    &
\end{pNiceMatrix}$
```

Typst

```
$ nicemat(first-row: #true, last-row: #true,
  first-col: #true, last-col: #true,
  dots: #(nullify: true),
  , C_1, dots.c, , C_4, ;
  L_1, a_11, a_12, a_13, a_14, L_1;
  dots.v, a_21, a_22, a_23, a_24, dots.v;
  , a_31, a_32, a_33, a_34, ;
  L_4, a_41, a_42, a_43, a_44, L_4;
  , C_1, dots.c, , C_4, ) $
```

$$\begin{array}{c} \\ L_1 \\ \vdots \\ L_4 \end{array} \begin{array}{c} \begin{array}{cccc} C_1 & \cdots\cdots & & C_4 \end{array} \\ \begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix} \\ \begin{array}{cccc} C_1 & \cdots\cdots & & C_4 \end{array} \end{array} \begin{array}{c} \\ L_1 \\ \vdots \\ L_4 \end{array}$$

LaTeX — nicematrix manual, p. 37

```
\NiceMatrixOptions{code-for-first-row =
\color{red},
                   code-for-first-col =
\color{blue},
                   code-for-last-row =
\color{green},
                   code-for-last-col =
\color{magenta}}

$\begin{pNiceArray}{cc|cc}[first-row,last-
row=5,first-col,last-col,xdots/nullify]
       & C_1    & \multicolumn1c{\Cdots} &        &
C_4    &        \\
L_1    & a_{11} & a_{12} & a_{13} & a_{14} & L_1
\\
\Vdots & a_{21} & a_{22} & a_{23} & a_{24} & \Vdots
\\
\hline
       & a_{31} & a_{32} & a_{33} & a_{34} &
\\
L_4    & a_{41} & a_{42} & a_{43} & a_{44} & L_4
\\
       & C_1    & \multicolumn1c{\Cdots} &        &
C_4    &
\end{pNiceArray}$
```

Typst

```
$ nicemat(first-row: #true, last-row: #true,
  first-col: #true, last-col: #true,
  vlines: 2, dots: #(nullify: true),
  map-cells: #((i, j, it) => {
    let c = if i == 0 { red } else if i == 5
{ green }
      else if j == 0 { blue } else if j == 5
{ fuchsia }
    if c == none { it } else { text(fill: c, it) }
  }),
  , C_1, cdots(fill: #red), , C_4, ;
  L_1, a_11, a_12, a_13, a_14, L_1;
  vdots(fill: #blue), a_21, a_22, a_23, a_24,
    vdots(fill: #fuchsia);
  hline();
  , a_31, a_32, a_33, a_34, ;
  L_4, a_41, a_42, a_43, a_44, L_4;
  , C_1, cdots(fill: #green), , C_4, ) $
```

$$\begin{array}{c} \\ L_1 \\ \vdots \\ L_4 \end{array} \begin{array}{c} \begin{array}{cccc} C_1 & \cdots\cdots & & C_4 \end{array} \\ \left(\begin{array}{cc:c:c} a_{11} & a_{12} & a_{13} & a_{14} \\ \hdashline a_{21} & a_{22} & a_{23} & a_{24} \\ \hdashline a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{array}\right) \\ \begin{array}{cccc} C_1 & \cdots\cdots & & C_4 \end{array} \end{array} \begin{array}{c} \\ L_1 \\ \vdots \\ L_4 \end{array}$$

`code-for-first-row` & co. become `map-cells`; the leaders get their color directly.

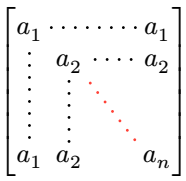
8 Dotted leaders in the matrices

LaTeX — nicematrix manual, p. 38

```
$\begin{bNiceMatrix}
a_1      & \Cdots &        & & a_1    \\
\Vdots   & a_2    & \Cdots & & a_2    \\
         & \Vdots & \Ddots[color=red] \\
\\
a_1      & a_2    &        & & a_n
\end{bNiceMatrix}$
```

Typst

```
$ nicemat(delim: "[",
  a_1, dots.c, , , a_1;
  dots.v, a_2, dots.c, , a_2;
  , dots.v, ddots(fill: #red);
  ;
  a_1, a_2, , , a_n) $
```



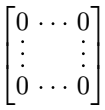
The dots symbols of Typst are the leaders: `dots.c`, `dots.h` (or `...`), `dots.v`, `dots.down`, `dots.up`; the functions `cdots(..)` & co. take options.

LaTeX — nicematrix manual, p. 38

```
$\begin{bNiceMatrix}
0      & \Cdots & 0      \\
\Vdots &        & \Vdots \\
0      & \Cdots & 0
\end{bNiceMatrix}$
```

Typst

```
$ nicemat(delim: "[",
  0, dots.c, 0;
  dots.v, , dots.v;
  0, dots.c, 0) $
```

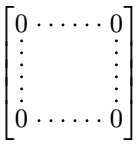


LaTeX — nicematrix manual, p. 38

```
$\begin{bNiceMatrix}
0      & \Cdots & \Cdots & 0      \\
\Vdots &        &        & \Vdots \\
\Vdots &        &        & \Vdots \\
0      & \Cdots & \Cdots & 0
\end{bNiceMatrix}$
```

Typst

```
$ nicemat(delim: "[",
  0, dots.c, dots.c, 0;
  dots.v, , , dots.v;
  dots.v, , , dots.v;
  0, dots.c, dots.c, 0) $
```

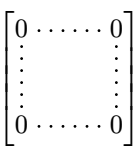


LaTeX — nicematrix manual, p. 38

```
$\begin{bNiceMatrix}
0      & \Cdots &        & 0      \\
\Vdots &        &        &        \\
       &        &        & \Vdots \\
0      &        & \Cdots & 0
\end{bNiceMatrix}$
```

Typst

```
$ nicemat(delim: "[",
  0, dots.c, , 0;
  dots.v, , , ;
  , , , dots.v;
  0, , dots.c, 0) $
```



LaTeX — nicematrix manual, p. 39

```
$\begin{bNiceMatrix}
0      & \Cdots & \Hspace*{1cm} & 0      \\
\Vdots &        &               & \Vdots \\[1cm]
0      & \Cdots &               & 0
\end{bNiceMatrix}$
```

Typst

```
$ nicemat(delim: "[", row-gap: #(0.2em, 1cm),
  0, dots.c, #h(1cm), 0;
  dots.v, , , dots.v;
  0, dots.c, , 0) $
```

$$\begin{bmatrix} 0 & \cdots & & 0 \\ \vdots & & & \vdots \\ & & & \\ 0 & \cdots & & 0 \end{bmatrix}$$

A cell holding only `#h(..)` counts as empty (`\Hspace`); `\\[1cm]` becomes a gap in `row-gap`.

LaTeX — nicematrix manual, p. 39

```
$A = \begin{pmatrix}
h & i & j & k & l & m \\
x &   &   &   &   & x
\end{pmatrix}$
```

Typst

```
$ A = mat(h, i, j, k, l, m;
  x, , , , , x) $
```

$$A = \begin{pmatrix} h & i & j & k & l & m \\ x & & & & & x \end{pmatrix}$$

Typst's own `mat`, for comparison.

LaTeX — nicematrix manual, p. 39

```
$B = \begin{pmatrix}
h & i & j & k & l & m \\
x & \ldots & \ldots & \ldots & \ldots & x
\end{pmatrix}$
```

Typst

```
$ B = mat(h, i, j, k, l, m;
  x, dots.h, dots.h, dots.h, dots.h, x) $
```

$$B = \begin{pmatrix} h & i & j & k & l & m \\ x & \ldots & \ldots & \ldots & \ldots & x \end{pmatrix}$$

Typst's own `mat`, for comparison.

LaTeX — nicematrix manual, p. 39

```
$C = \begin{pNiceMatrix}
h & i & j & k & l & m \\
x & \Ldots & \Ldots & \Ldots & \Ldots & x
\end{pNiceMatrix}$
```

Typst

```
$ C = nicemat(h, i, j, k, l, m;
  x, dots.h, dots.h, dots.h, dots.h, x) $
```

$$C = \begin{pNiceMatrix} h & i & j & k & l & m \\ x & \Ldots & & & & x \end{pNiceMatrix}$$

LaTeX — nicematrix manual, p. 39

```
$D = \begin{pNiceMatrix}[xdots/nullify]
h & i & j & k & l & m \\
x & \Ldots & & & & x
\end{pNiceMatrix}$
```

Typst

```
$ D = nicemat(dots: #(nullify: true),
  h, i, j, k, l, m;
  x, dots.h, , , , x) $
```

$$D = \begin{pNiceMatrix} h & i & j & k & l & m \\ x & \Ldots & & & & x \end{pNiceMatrix}$$

LaTeX — nicematrix manual, p. 40

```latex
$\begin{pNiceMatrix}
1 & 2 & 3 & 4 & 5 \\
1 & \Hdotsfor{3} & 5 \\
1 & 2 & 3 & 4 & 5 \\
1 & 2 & 3 & 4 & 5
\end{pNiceMatrix}$
```

Typst

```
$ nicemat(
  1, 2, 3, 4, 5;
  1, cdots(span: #3), 5;
  1, 2, 3, 4, 5;
  1, 2, 3, 4, 5) $
```

$$\begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ 1 & \cdots & \cdots & \cdots & 5 \\ 1 & 2 & 3 & 4 & 5 \\ 1 & 2 & 3 & 4 & 5 \end{pmatrix}$$

`\Hdotsfor{3}` is a leader with a `span`.

LaTeX — nicematrix manual, p. 40

```latex
$\begin{pNiceMatrix}
1 & 2 & 3 & 4 & 5 \\
  & \Hdotsfor{3} \\
1 & 2 & 3 & 4 & 5 \\
1 & 2 & 3 & 4 & 5
\end{pNiceMatrix}$
```

Typst

```
$ nicemat(
  1, 2, 3, 4, 5;
  , cdots(span: #3);
  1, 2, 3, 4, 5;
  1, 2, 3, 4, 5) $
```

$$\begin{pmatrix} 1 & 2 & 3 & 4 & 5 \\ & \cdots & \cdots & \cdots & \\ 1 & 2 & 3 & 4 & 5 \\ 1 & 2 & 3 & 4 & 5 \end{pmatrix}$$

LaTeX — nicematrix manual, p. 40

```
$\begin{bNiceMatrix}
C[a_1,a_1] & \Cdots & C[a_1,a_n]
  & \hspace*{20mm} & C[a_1,a_1^{(p)}] & \Cdots &
C[a_1,a_n^{(p)}] \\
\Vdots & \Ddots & \Vdots
  & \Hdotsfor{1} & \Vdots & \Ddots & \Vdots \\
C[a_n,a_1] & \Cdots & C[a_n,a_n]
  & & C[a_n,a_1^{(p)}] & \Cdots & C[a_n,a_n^{(p)}]
\\
\rule{0pt}{15mm}\NotEmpty  & \Vdotsfor{1} & &
\Ddots & & \Vdotsfor{1} \\
C[a_1^{(p)},a_1] & \Cdots & C[a_1^{(p)},a_n]
  & & C[a_1^{(p)},a_1^{(p)}] & \Cdots &
C[a_1^{(p)},a_n^{(p)}] \\
\Vdots & \Ddots & \Vdots
  & \Hdotsfor{1} & \Vdots & \Ddots & \Vdots \\
C[a_n^{(p)},a_1] & \Cdots & C[a_n^{(p)},a_n]
  & & C[a_n^{(p)},a_1^{(p)}] & \Cdots &
C[a_n^{(p)},a_n^{(p)}]
\end{bNiceMatrix}$
```

Typst

```
#set text(size: 9pt)
$ nicemat(delim: "[",
  C[a_1\, a_1], dots.c, C[a_1\, a_n], #h(20mm),
    C[a_1\, a_1^((p))], dots.c, C[a_1\, a_n^((p))];
  dots.v, dots.down, dots.v, cdots(span: #1),
    dots.v, dots.down, dots.v;
  C[a_n\, a_1], dots.c, C[a_n\, a_n], ,
    C[a_n\, a_1^((p))], dots.c, C[a_n\, a_n^((p))];
  cell(#box(height: 15mm), empty: #false),
    vdots(span: #1), , dots.down, , vdots(span:
#1);
  C[a_1^((p))\, a_1], dots.c, C[a_1^((p))\, a_n], ,
    C[a_1^((p))\, a_1^((p))], dots.c,
    C[a_1^((p))\, a_n^((p))];
  dots.v, dots.down, dots.v, cdots(span: #1),
    dots.v, dots.down, dots.v;
  C[a_n^((p))\, a_1], dots.c, C[a_n^((p))\, a_n], ,
    C[a_n^((p))\, a_1^((p))], dots.c,
    C[a_n^((p))\, a_n^((p))]) $
```

$$\begin{bmatrix}
C[a_1,a_1] & \cdots & C[a_1,a_n] & & C\left[a_1,a_1^{(p)}\right] & \cdots & C\left[a_1,a_n^{(p)}\right] \\
\vdots & \ddots & \vdots & \cdots & \vdots & \ddots & \vdots \\
C[a_n,a_1] & \cdots & C[a_n,a_n] & & C\left[a_n,a_1^{(p)}\right] & \cdots & C\left[a_n,a_n^{(p)}\right] \\
 & \iddots & & \iddots & & \iddots & \\
C\left[a_1^{(p)},a_1\right] & \cdots & C\left[a_1^{(p)},a_n\right] & & C\left[a_1^{(p)},a_1^{(p)}\right] & \cdots & C\left[a_1^{(p)},a_n^{(p)}\right] \\
\vdots & \ddots & \vdots & \cdots & \vdots & \ddots & \vdots \\
C\left[a_n^{(p)},a_1\right] & \cdots & C\left[a_n^{(p)},a_n\right] & & C\left[a_n^{(p)},a_1^{(p)}\right] & \cdots & C\left[a_n^{(p)},a_n^{(p)}\right]
\end{bmatrix}$$

A comma inside a cell is escaped with `\,`. `\Vdotsfor{1}` is `vdots(span: 1)`; `\rule{0pt}{15mm}\NotEmpty` is a tall box in a cell forced to count as non-empty.

LaTeX — nicematrix manual, p. 41

```
\NiceMatrixOptions{renew-dots,renew-matrix}
$\begin{pmatrix}
1      & \cdots & \cdots & 1      \\
0      & \ddots &        & \vdots \\
\vdots & \ddots & \ddots & \vdots \\
0      & \cdots & 0      & 1
\end{pmatrix}$
```

Typst

```
#show math.mat: nicemat
$ mat(
  1, dots.c, dots.c, 1;
  0, dots.down, , dots.v;
  dots.v, dots.down, dots.down, dots.v;
  0, dots.c, 0, 1) $
```

$$\begin{pmatrix}
1 & \cdots & \cdots & 1 \\
0 & \ddots & & \vdots \\
\vdots & \ddots & \ddots & \vdots \\
0 & \cdots & 0 & 1
\end{pmatrix}$$

`renew-matrix` and `renew-dots` are a single show rule: every `mat` becomes a `nicemat`, with the same geometry.

LaTeX — nicematrix manual, p. 41

```
$\begin{bNiceMatrix}
1 & \hspace*{1cm}            & 0 \\[8mm]
  & \Ddots^{n \text{ times}} &   \\
0 &                          & 1
\end{bNiceMatrix}$
```

Typst

```
$ nicemat(delim: "[", row-gap: #(8mm, 0.2em),
  1, #h(1cm), 0;
  , ddots(above: n "times"), ;
  0, , 1) $
```

$$\begin{bmatrix} 1 & & 0 \\ & \overset{n \text{ times}}{\ddots} & \\ 0 & & 1 \end{bmatrix}$$

The labels `^`, `_` and `:` are `above`, `below` and `middle`.

LaTeX — nicematrix manual, p. 41

```
$\begin{bNiceMatrix}
1 & \hspace*{1cm}            & 0 \\[8mm]
  & \Ddots[horizontal-label]^{n \text{ times}} &
\\
0 &                          & 1
\end{bNiceMatrix}$
```

Typst

```
$ nicemat(delim: "[", row-gap: #(8mm, 0.2em),
  1, #h(1cm), 0;
  , ddots(above: n "times",
      horizontal-labels: #true), ;
  0, , 1) $
```

$$\begin{bmatrix} 1 & & 0 \\ & \overset{n \text{ times}}{\ddots} & \\ 0 & & 1 \end{bmatrix}$$

LaTeX — nicematrix manual, p. 43

```
$\begin{pNiceMatrix}[xdots={nullify, shorten +=
2pt, line-style = loosely dotted}]
a      & b      & 0      &        & \Cdots & 0
\\
b      & a      & b      & \Ddots &        & \Vdots
\\
0      & b      & a      & \Ddots &        &
\\
       & \Ddots & \Ddots & \Ddots &        & 0
\\
\Vdots &        &        &        &        & b
\\
0      & \Cdots &        & 0      & b      & a
\end{pNiceMatrix}$
```

Typst

```
$ nicemat(dots: #(nullify: true,
    shorten: 0.3em + 2pt,
    stroke: (dash: "loosely-dotted")),
  a, b, 0, , dots.c, 0;
  b, a, b, dots.down, , dots.v;
  0, b, a, dots.down, , ;
  , dots.down, dots.down, dots.down, , 0;
  dots.v, , , , , b;
  0, dots.c, , 0, b, a) $
```

$$\begin{pmatrix} a & b & 0 & \cdots & & 0 \\ b & a & b & \ddots & & \vdots \\ 0 & b & a & \ddots & \ddots & \\ \vdots & \ddots & \ddots & \ddots & \ddots & 0 \\ \vdots & & \ddots & \ddots & \ddots & b \\ 0 & \cdots & & 0 & b & a \end{pmatrix}$$

`line-style` becomes `stroke`: `auto` gives the round dots of nicematrix, any Typst stroke gives a line.

LaTeX — nicematrix manual, p. 43

```
$\begin{bNiceMatrix}[margin,hvlines]
\Block{3-3}<\LARGE>{A} & & & 0 \\
& \hspace*{1cm} & & \Vdots \\
& & & 0 \\
0 & \Cdots& 0 & 0
\end{bNiceMatrix}$
```

Typst

```
$ nicemat(delim: "[", margin: #(x: 5pt),
  hlines: #true, vlines: #true,
  column-width: #(auto, 1cm),
  cell(#text(1.728em)[$A$],
    rowspan: #3, colspan: #3), 0;
  dots.v;
  0;
  0, dots.c, 0, 0) $
```

$$\left[\begin{array}{c|c} \multicolumn{1}{c}{\multirow{3}{*}{A}} & \begin{matrix}0\\\vdots\\0\end{matrix} \\ \cline{2-2} 0 \cdots\cdots 0 & 0 \end{array}\right]$$

The rules are not drawn in the block nor across the leaders. The `\hspace*{1cm}` written under the block becomes a column width.

LaTeX — nicematrix manual, p. 44

```
$\begin{NiceArray}{ccccc}%
  [ hvlines ,
    first-row ,
    last-row = 6,
    first-col ,
    last-col ,
    xdots/horizontal-labels ]
& \Hbrace{3}{p} & \Hbrace[brace-shift=1mm]{2}{q} \\
\Vbrace{3}{p} & 1 & 1 & 134 & 1 & 1 & \Vbrace{3}{p}
\\
& 1 & 1 & 134 & 1 & 1 \\
& 1 & 1 & 13456 & 1 & 1 \\
\Vbrace{2}{q} & 1 & 1 & 134 & 1 & 1 & \Vbrace{2}
{q}\\
& 1 & 1 & 134 & 1 & 1 \\
& \Hbrace{3}{p} & \Hbrace[color=blue]{2}{q} \\
\end{NiceArray}$
```

Typst

```
$ nicearray(first-row: #true,
  last-row: #true, first-col: #true,
  last-col: #true, hlines: #true,
  vlines: #true,
  , hbrace(p, span: #3),
    hbrace(q, span: #2, shift: #1mm);
  vbrace(p, span: #3), 1, 1, 134, 1, 1,
    vbrace(p, span: #3);
  1, 1, 134, 1, 1;
  1, 1, 13456, 1, 1;
  vbrace(q, span: #2), 1, 1, 134, 1, 1,
    vbrace(q, span: #2);
  1, 1, 134, 1, 1;
  , hbrace(p, span: #3),
    hbrace(q, span: #2, fill: #blue)) $
```

$$\begin{array}{c} \overbrace{}^{p}\ \overbrace{}^{q} \\ \left.\begin{array}{|cc|c|cc|}\hline 1&1&134&1&1\\ 1&1&134&1&1\\ 1&1&13456&1&1\\ \hline 1&1&134&1&1\\ 1&1&134&1&1\\ \hline\end{array}\right. \\ \underbrace{}_{p}\ \underbrace{}_{q} \end{array}$$

The braces cover the rows below them, so those rows start directly with their first cell. They are drawn with the braces of the math font (TikZ decorations have no equivalent).

9 Delimiters in the preamble of the environment

LaTeX — nicematrix manual, p. 45

```
$\begin{NiceArray}{\left\lgroup ccc\right\rgroup l}
1 & 2 & 3 &
4 & 1 & 6 &
7 & 8 & 9 & \scriptstyle L_3 \gets L_3 + L_1 + L_2
\end{NiceArray}$
```

Typst

```
$ nicearray(
  1, 2, 3, ;
  4, 1, 6, ;
  7, 8, 9, script(L_3 <- L_3 + L_1 + L_2),
  submatrix((0, 0), (2, 2), delim: "(")) $
```

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 1 & 6 \\ 7 & 8 & 9 \end{pmatrix} \quad L_3 \leftarrow L_3 + L_1 + L_2$$

Delimiters in the preamble are sub-matrices; they make room for themselves.

LaTeX — nicematrix manual, p. 45

```
$\begin{pNiceArray}{(c)(c)(c)}
a_{11} & a_{12}
& a_{13} \\
a_{21} & \displaystyle \int_0^1\dfrac{1}{x^2+1}\,dx
& a_{23} \\
a_{31} & a_{32}
& a_{33}
\end{pNiceArray}$
```

Typst

```
$ nicemat(
  a_11, a_12, a_13;
  a_21, display(integral_0^1 1/(x^2 + 1) dif x),
    a_23;
  a_31, a_32, a_33,
  submatrix((0, 0), (2, 0)),
  submatrix((0, 1), (2, 1)),
  submatrix((0, 2), (2, 2))) $
```

$$\left(\begin{pmatrix} a_{11} \\ a_{21} \\ a_{31} \end{pmatrix} \begin{pmatrix} a_{12} \\ \displaystyle\int_0^1 \frac{1}{x^2+1}\,\mathrm{d}x \\ a_{32} \end{pmatrix} \begin{pmatrix} a_{13} \\ a_{23} \\ a_{33} \end{pmatrix} \right)$$

10 The \CodeAfter

LaTeX — nicematrix manual, p. 46

```
$\begin{pNiceMatrix}[xdots/shorten += 0.3 em]
  I       & 0       & \Cdots  &0      \\
  0       & I       & \Ddots  &\Vdots\\
  \Vdots  &\Ddots   & I       &0      \\
  0       &\Cdots   & 0       &I
\CodeAfter \line{2-2}{3-3}
\end{pNiceMatrix}$
```

Typst

```
$ nicemat(
  I, 0, dots.c, 0;
  0, I, dots.down, dots.v;
  dots.v, dots.down, I, 0;
  0, dots.c, 0, I,
  dotline((1, 1), (2, 2))) $
```

$$\begin{pmatrix} I & 0 & \cdots & 0 \\ 0 & I & \ddots & \vdots \\ \vdots & \ddots & I & 0 \\ 0 & \cdots & 0 & I \end{pmatrix}$$

`\line` becomes the `dotline` decoration. The shortening is left at its default: Typst's matrices are tighter than LaTeX's and `shorten += 0.3em` would leave no room for the dots.

LaTeX — nicematrix manual, p. 46

```
$\begin{bNiceMatrix}
  1      & \Cdots &   & 1      & 2      & \Cdots
& 2      \\
  0      & \Ddots &   & \Vdots & \Vdots &
\hspace*{2.5cm} & \Vdots \\
  \Vdots & \Ddots &   &        &        &
&        \\
  0      & \Cdots & 0 & 1      & 2      & \Cdots
& 2
\CodeAfter \line[shorten=6pt]{1-5}{4-7}
\end{bNiceMatrix}$
```

Typst

```
$ nicemat(delim: "[",
  1, dots.c, , 1, 2, dots.c, 2;
  0, dots.down, , dots.v, dots.v, #h(2.5cm),
    dots.v;
  dots.v, dots.down, , , , , ;
  0, dots.c, 0, 1, 2, dots.c, 2,
  dotline((0, 4), (3, 6), shorten: #6pt)) $
```

$$\begin{bmatrix} 1 & \cdots & 1 & 2 & \cdots & 2 \\ 0 & \ddots & \vdots & \vdots & \iddots & \vdots \\ \vdots & \iddots & \vdots & \vdots & \ddots & \vdots \\ 0 & \cdots & 0 & 1 & 2 & 2 \end{bmatrix}$$

LaTeX — nicematrix manual, p. 47

```
$\begin{NiceArray}{ccc@{\hspace{1.5em}}c}[cell-
space-limits += 2pt,margin]
   1          & 1            & 1            & x \\
  \dfrac{1}{4} & \dfrac{1}{2} & \dfrac{1}{4} & y \\
   1          & 2            & 3            & z
\CodeAfter
  \SubMatrix({1-1}{3-3})
  \SubMatrix({1-4}{3-4})
\end{NiceArray}$
```

Typst

```
$ nicearray(cell-space: #2pt,
  1, 1, 1, x;
  display(1/4), display(1/2), display(1/4), y;
  1, 2, 3, z,
  submatrix((0, 0), (2, 2)),
  submatrix((0, 3), (2, 3))) $
```

$$\begin{pmatrix} 1 & 1 & 1 \\ \frac{1}{4} & \frac{1}{2} & \frac{1}{4} \\ 1 & 2 & 3 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

No manual spacing is needed: a sub-matrix makes room for its delimiters (`reserve: #false` to opt out).

LaTeX — nicematrix manual, p. 47

```
$\begin{NiceArray}{(ccc)(c)}[cell-space-limits +=
2pt]
   1          & 1            & 1            & x \\
  \dfrac{1}{4} & \dfrac{1}{2} & \dfrac{1}{4} & y \\
   1          & 2            & 3            & z
\end{NiceArray}$
```

Typst

```
$ nicearray(cell-space: #2pt,
  1, 1, 1, x;
  display(1/4), display(1/2), display(1/4), y;
  1, 2, 3, z,
  submatrix((0, 0), (2, 2)),
  submatrix((0, 3), (2, 3))) $
```

$$\begin{pmatrix} 1 & 1 & 1 \\ \frac{1}{4} & \frac{1}{2} & \frac{1}{4} \\ 1 & 2 & 3 \end{pmatrix} \begin{pmatrix} x \\ y \\ z \end{pmatrix}$$

The preamble `(ccc)(c)` is the same thing as the two sub-matrices above.

LaTeX — nicematrix manual, p. 47

```
$\begin{bNiceMatrix}[right-margin=1em]
  1 & 1 & 1 \\
  1 & a & b \\
  1 & c & d
\CodeAfter
  \SubMatrix[{2-2}{3-3}]^{T}
\end{bNiceMatrix}$
```

Typst

```
$ nicemat(delim: "[",
  1, 1, 1;
  1, a, b;
  1, c, d,
  submatrix((1, 1), (2, 2), delim: "[",
    sup: T)) $
```

$$\begin{bmatrix} 1 & 1 & 1 \\ 1 & \begin{bmatrix} a & b \\ c & d \end{bmatrix}^T \end{bmatrix}$$

LaTeX — nicematrix manual, p. 48

```
$\begin{NiceArray}{cc@{\hspace{5mm}}l}[cell-space-
limits=2pt]}
    &   & \frac{1}{2} \\
    &   & \frac{1}{4} \\[1mm]
  a & b & \frac{1}{2}a+\frac{1}{4}b \\
  c & d & \frac{1}{2}c+\frac{1}{4}d \\
\CodeAfter
  \SubMatrix({1-3}{2-3})
  \SubMatrix({3-1}{4-2})
  \SubMatrix({3-3}{4-3})
\end{NiceArray}$
```

Typst

```
$ nicearray(cell-space: #2pt,
  align: #(center, center, left),
  column-gap: #(0.5em, 5mm),
  row-gap: #(0.2em, 1mm + 0.2em, 0.2em),
  , , 1/2;
  , , 1/4;
  a, b, 1/2 a + 1/4 b;
  c, d, 1/2 c + 1/4 d,
  submatrix((0, 2), (1, 2)),
  submatrix((2, 0), (3, 1)),
  submatrix((2, 2), (3, 2))) $
```

$$\begin{array}{cc@{\hspace{5mm}}l} & & \frac{1}{2} \\ & & \frac{1}{4} \\ a & b & \frac{1}{2}a+\frac{1}{4}b \\ c & d & \frac{1}{2}c+\frac{1}{4}d \end{array}$$

LaTeX — nicematrix manual, p. 48

```
$\begin{NiceArray}{cc@{\hspace{5mm}}l}[cell-space-
limits=2pt]}
    &   & \frac{1}{2} \\
    &   & \frac{1}{4} \\[1mm]
  a & b & \frac{1}{2}a+\frac{1}{4}b \\
  c & d & \frac{1}{2}c+\frac{1}{4}d \\
\CodeAfter
  \SubMatrix({1-3}{2-3})[slim]
  \SubMatrix({3-1}{4-2})
  \SubMatrix({3-3}{4-3})
\end{NiceArray}$
```

Typst

```
$ nicearray(cell-space: #2pt,
  align: #(center, center, left),
  column-gap: #(0.5em, 5mm),
  row-gap: #(0.2em, 1mm + 0.2em, 0.2em),
  , , 1/2;
  , , 1/4;
  a, b, 1/2 a + 1/4 b;
  c, d, 1/2 c + 1/4 d,
  submatrix((0, 2), (1, 2), slim: #true),
  submatrix((2, 0), (3, 1)),
  submatrix((2, 2), (3, 2))) $
```

$$\begin{array}{cc@{\hspace{5mm}}l} & & \frac{1}{2} \\ & & \frac{1}{4} \\ a & b & \frac{1}{2}a+\frac{1}{4}b \\ c & d & \frac{1}{2}c+\frac{1}{4}d \end{array}$$

LaTeX — nicematrix manual, p. 49

```
$\begin{NiceArray}{ccc@{}w{c}{5mm}@{}ccc}
    &   &   && -1 & 1  & 2  \\
    &   &   &&  0 & 3  & 4  \\
    &   &   &&  0 & 0  & 5  \\
  1 & 2 & 3 && -1 & 7  & 25 \\
  0 & 4 & 5 && 0  & 12 & 41 \\
  0 & 0 & 6 && 0  & 0  & 30
\CodeAfter
  \NewDocumentCommand{\MyDraw}{}{\tikz \draw [blue]
(2-|1) -| (3-|2) -| (4-|3) ;}
  \SubMatrix({1-5}{3-7})[code = \MyDraw ]
  \SubMatrix({4-1}{6-3})[code = \MyDraw ]
  \SubMatrix({4-5}{6-7})[code = \MyDraw ]
\end{NiceArray}$
```

Typst

```
// Rules under the diagonal of the 3×3 sub-matrix
at (i, j),
// like `(2-|1) -| (3-|2) -| (4-|3)` relative to
the sub-matrix.
#let stairs(g, i, j) = {
  let y(k) = if k < g.rows.len()
{ g.rows.at(k).tile-top } else
{ g.rows.last().tile-bottom }
  // The left edge of the sub-matrix is just inside
its delimiter.
  let x(k) = if k == j { g.cols.at(k).left - 2pt }
else { g.cols.at(k).tile-left }
  place(curve(stroke: blue,
    curve.move((x(j), y(i + 1))),
    curve.line((x(j + 1), y(i + 1))),
curve.line((x(j + 1), y(i + 2))),
    curve.line((x(j + 2), y(i + 2))),
curve.line((x(j + 2), y(i + 3)))))
}
$ nicearray(
  column-width: #(auto, auto, auto, 5mm),
  column-gap: #(0.5em, 0.5em, 0pt, 0pt, 0.5em,
0.5em),
  foreground: #(g => {
    stairs(g, 0, 4); stairs(g, 3, 0); stairs(g, 3,
4) }),
  , , , , -1, 1, 2;
  , , , , 0, 3, 4;
  , , , , 0, 0, 5;
  1, 2, 3, , -1, 7, 25;
  0, 4, 5, , 0, 12, 41;
  0, 0, 6, , 0, 0, 30,
  submatrix((0, 4), (2, 6)),
  submatrix((3, 0), (5, 2)),
  submatrix((3, 4), (5, 6))) $
```

$$\begin{pmatrix} 1 & 2 & 3 \\ 0 & 4 & 5 \\ 0 & 0 & 6 \end{pmatrix} \qquad \begin{pmatrix} -1 & 1 & 2 \\ 0 & 3 & 4 \\ 0 & 0 & 5 \end{pmatrix} \qquad \begin{pmatrix} -1 & 7 & 25 \\ 0 & 12 & 41 \\ 0 & 0 & 30 \end{pmatrix}$$

The `code` key of `\SubMatrix` (TikZ code relative to the sub-matrix) becomes a function using the geometry of the matrix.

LaTeX — nicematrix manual, p. 50

```
$\begin{pNiceMatrix}
  1  & 2  & 3  & 4  & 5  & 6  \\
  11 & 12 & 13 & 14 & 15 & 16 \\
\CodeAfter
  \OverBrace{1-1}{2-3}{A}
  \OverBrace{1-4}{2-6}{B}
\end{pNiceMatrix}$
```

Typst

```
$ nicemat(
  1, 2, 3, 4, 5, 6;
  11, 12, 13, 14, 15, 16,
  hbrace((0, 0), (1, 2), A),
  hbrace((0, 3), (1, 5), B)) $
```

$$\begin{pmatrix} \overbrace{1}^{A} & 2 & 3 & \overbrace{4}^{B} & 5 & 6 \\ 11 & 12 & 13 & 14 & 15 & 16 \end{pmatrix}$$

`\OverBrace` is `hbrace(from, to, label)` (and `\UnderBrace` has `side: bottom`). Room is made for the braces.

LaTeX — nicematrix manual, p. 50

```
$\begin{pNiceMatrix}
  1  & 2  & 3  & 4  & 5  & 6  \\
  11 & 12 & 13 & 14 & 15 & 16 \\
\CodeAfter
  \OverBrace[shorten,yshift=3pt]{1-1}{2-3}{A}
  \OverBrace[shorten,yshift=3pt]{1-4}{2-6}{B}
\end{pNiceMatrix}$
```

Typst

```
$ nicemat(
  1, 2, 3, 4, 5, 6;
  11, 12, 13, 14, 15, 16,
  hbrace((0, 0), (1, 2), A,
    shorten: #true, shift: #3pt),
  hbrace((0, 3), (1, 5), B,
    shorten: #true, shift: #3pt)) $
```

$$\overbrace{}^{A}\ \overbrace{}^{B}$$
$$\begin{pmatrix} 1 & 2 & 3 & 4 & 5 & 6 \\ 11 & 12 & 13 & 14 & 15 & 16 \end{pmatrix}$$

11 Other features

LaTeX — nicematrix manual, p. 59

```
$\begin{pNiceArray}{ScW{c}{1cm}c}[xdots/
nullify,first-row]
  {C_1} & \Cdots &  & C_n \\
  2.3  & 0 & \Cdots & 0 \\
  12.4 & \Vdots & & \Vdots \\
  1.45 \\
  7.2  & 0 & \Cdots & 0
\end{pNiceArray}$
```

Typst

```
$ nicemat(first-row: #true,
  dots: #(nullify: true),
  align: #(right, center),
  column-width: #(auto, auto, 1cm),
  C_1, dots.c, , C_n;
  2.3 #hide[$0$], 0, dots.c, 0;
  12.4 #hide[$0$], dots.v, , dots.v;
  1.45;
  7.2 #hide[$0$], 0, dots.c, 0) $
```

$$\begin{pmatrix} C_1 & \cdots & C_n \\ 2.3 & 0 & \cdots & 0 \\ 12.4 & \vdots & & \vdots \\ 1.45 & \vdots & & \vdots \\ 7.2 & 0 & \cdots & 0 \end{pmatrix}$$

Typst has no column of type `S`: the decimal points are aligned here with right alignment and a hidden digit.

LaTeX — nicematrix manual, p. 59

```
$\begin{bNiceMatrix}[r]
  \cos x & - \sin x \\
  \sin x & \cos x
\end{bNiceMatrix}$
```

Typst

```
$ nicemat(delim: "[", align: #right,
  cos x, -sin x;
  sin x, cos x) $
```

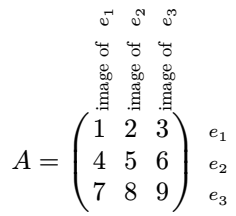
$$\begin{bmatrix} \cos x & -\sin x \\ \sin x & \cos x \end{bmatrix}$$

LaTeX — nicematrix manual, p. 60

```
\NiceMatrixOptions
 {
   code-for-first-row = \scriptstyle \rotate
\text{image of },
   code-for-last-col = \scriptstyle
 }
$A = \begin{pNiceMatrix}[first-row,last-col=4]
  e_1 & e_2 & e_3       \\
  1   & 2   & 3   & e_1 \\
  4   & 5   & 6   & e_2 \\
  7   & 8   & 9   & e_3
\end{pNiceMatrix}$
```

Typst

```
$ A = nicemat(first-row: #true,
  last-col: #true,
  map-cells: #((i, j, it) => if i == 0 {
      rotate(-90deg, reflow: true,
        $script("image of " #it)$)
    } else if j == 3 { $script(#it)$ }
    else { it }),
  e_1, e_2, e_3;
  1, 2, 3, e_1;
  4, 5, 6, e_2;
  7, 8, 9, e_3) $
```



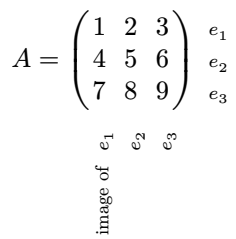
`\rotate` is Typst's `rotate`.

LaTeX — nicematrix manual, p. 60

```
\NiceMatrixOptions
  {
    code-for-last-row = \scriptstyle \rotate ,
    code-for-last-col = \scriptstyle
  }
$A = \begin{pNiceMatrix}[last-row=4,last-col=4]
  1   & 2   & 3   & e_1 \\
  4   & 5   & 6   & e_2 \\
  7   & 8   & 9   & e_3 \\
  \text{image of } e_1 & e_2 & e_3
\end{pNiceMatrix}$
```

Typst

```
$ A = nicemat(last-row: #true,
  last-col: #true,
  map-cells: #((i, j, it) => if i == 3 {
      box(baseline: 100%, rotate(-90deg,
        reflow: true, $script(#it)$))
    } else if j == 3 { $script(#it)$ }
    else { it }),
  1, 2, 3, e_1;
  4, 5, 6, e_2;
  7, 8, 9, e_3;
  "image of " e_1, e_2, e_3) $
```



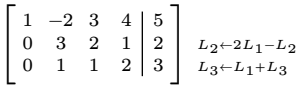
`box(baseline: 100%)` hangs the rotated labels from the top of the row.

LaTeX — nicematrix manual, p. 61

```
$\begin{bNiceArray}{cccc|c}[small,
                            last-col ,
                            code-for-last-col =
\scriptscriptstyle,
                            columns-width = 3mm ]
  1 & -2 & 3 & 4 & 5 \\
  0 & 3  & 2 & 1 & 2 & L_2 \gets 2 L_1 - L_2 \\
  0 & 1  & 1 & 2 & 3 & L_3 \gets L_1 + L_3
\end{bNiceArray}$
```

Typst

```
$ nicemat(delim: "[", small: #true,
  last-col: #true, vlines: 4,
  column-width: #3mm,
  map-cells: #((i, j, it) =>
    if j == 5 { $sscript(#it)$ } else { it }),
  1, -2, 3, 4, 5, ;
  0, 3, 2, 1, 2, L_2 <- 2 L_1 - L_2;
  0, 1, 1, 2, 3, L_3 <- L_1 + L_3) $
```



LaTeX — nicematrix manual, p. 61

```
$\begin{pNiceMatrix}% don't forget the %
    [first-row,
     first-col,
     code-for-first-row = \mathbf{\alph{jCol}} ,
     code-for-first-col = \mathbf{\arabic{iRow}} ]
  &   &    &    &   \\
  & 1 & 2  & 3  & 4 \\
  & 5 & 6  & 7  & 8 \\
  & 9 & 10 & 11 & 12
\end{pNiceMatrix}$
```

Typst

```
$ nicemat(first-row: #true, first-col: #true,
  map-cells: #((i, j, it) =>
    if i == 0 and j > 0 { $bold(#numbering("a",
j))$ }
    else if j == 0 and i > 0 { $bold(#str(i))$ }
    else { it }),
  , , , , ;
  , 1, 2, 3, 4;
  , 5, 6, 7, 8;
  , 9, 10, 11, 12) $
```

$$\begin{array}{c} \\ \mathbf{1} \\ \mathbf{2} \\ \mathbf{3} \end{array} \begin{array}{c} \begin{array}{cccc} \mathbf{a} & \mathbf{b} & \mathbf{c} & \mathbf{d} \end{array} \\ \begin{pmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \end{pmatrix} \end{array}$$

The counters `iRow` and `jCol` are the arguments of `map-cells`.

LaTeX — nicematrix manual, p. 62

```
$C = \pAutoNiceMatrix{3-3}{C_{\arabic{iRow},
\arabic{jCol}}}$
```

Typst

```
$ C = #nicemat(..range(3).map(i =>
  range(3).map(j => $C_(#(i + 1), #(j + 1))$))) $
```

$$C = \begin{pmatrix} C_{1,1} & C_{1,2} & C_{1,3} \\ C_{2,1} & C_{2,2} & C_{2,3} \\ C_{3,1} & C_{3,2} & C_{3,3} \end{pmatrix}$$

`\AutoNiceMatrix` is an array built in code.

LaTeX — nicematrix manual, p. 62

```
$\begin{bNiceMatrix}[light-syntax,first-row,first-
col]
  {} a             b                 ;
  a  2\cos a       {\cos a + \cos b} ;
  b \cos a+\cos b  { 2 \cos b }
\end{bNiceMatrix}$
```

Typst

```
$ nicemat(delim: "[",
  first-row: #true, first-col: #true,
  , a, b;
  a, 2 cos a, cos a + cos b;
  b, cos a + cos b, 2 cos b) $
```

$$\begin{array}{c} \\ a \\ b \end{array} \begin{array}{c} \begin{array}{cc} a & b \end{array} \\ \begin{bmatrix} 2\cos a & \cos a + \cos b \\ \cos a + \cos b & 2\cos b \end{bmatrix} \end{array}$$

`light-syntax` has no purpose: the syntax of Typst is already light.

LaTeX — nicematrix manual, p. 62

```
$\begin{bNiceMatrix}[delimiters/color=red]
  1 & 2 \\
  3 & 4
\end{bNiceMatrix}$
```

Typst

```
$ nicemat(delim: "[", delim-fill: #red,
  1, 2;
  3, 4) $
```

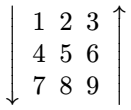
$$\begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}$$

LaTeX — nicematrix manual, p. 63

```
$\begin{NiceArrayWithDelims}
   {\downarrow}{\uparrow}{ccc}[margin]
  1 & 2 & 3 \\
  4 & 5 & 6 \\
  7 & 8 & 9
\end{NiceArrayWithDelims}$
```

Typst

```
$ nicemat(delim: #("↓", "↑"),
  margin: #(x: 5pt),
  1, 2, 3;
  4, 5, 6;
  7, 8, 9) $
```



`{NiceArrayWithDelims}` is `delim` with a pair; any extensible symbol can be used.

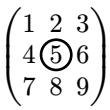
12 Explicit use of TikZ with nicematrix

LaTeX — nicematrix manual, p. 64

```
$\begin{pNiceMatrix}[name=mymatrix]
  1 & 2 & 3 \\
  4 & 5 & 6 \\
  7 & 8 & 9
\end{pNiceMatrix}$
\tikz[remember picture,overlay]
     \draw (mymatrix-2-2) circle (2mm) ;
```

Typst

```
$ nicemat(foreground: #(g => {
    let c = g.cells.at(1).at(1).ink
    place(dx: c.x + c.width / 2 - 2mm,
      dy: c.y + c.height / 2 - 2mm,
      circle(radius: 2mm))
  }),
  1, 2, 3;
  4, 5, 6;
  7, 8, 9) $
```



The TikZ nodes (named, or used in the `\CodeAfter`, p. 55) are the geometry given to `foreground`: `ink` is the content of a cell, `x`, `y`, `width`, `height` its tile.

LaTeX — nicematrix manual, p. 67

```
$\begin{NiceArray}{c@{\;}c@{\;}c@{\;}c@{\;}c}
[create-medium-nodes]
   u_1 &-& u_0 &=& r     \\
   u_2 &-& u_1 &=& r     \\
   u_3 &-& u_2 &=& r     \\
   u_4 &-& u_3 &=& r     \\
   \phantom{u_5} & &  \phantom{u_4}
&\smash{\vdots} &       \\
   u_n &-& u_{n-1} &=& r \\[3pt]
   \hline
   u_n &-& u_0 &=& nr \\
\CodeAfter
   \tikz[very thick, red, opacity=0.4, name suffix
= -medium]
     \draw (1-1.north west) -- (2-3.south east)
       (2-1.north west) -- (3-3.south east)
       (3-1.north west) -- (4-3.south east)
       (4-1.north west) -- (5-3.south east)
       (5-1.north west) -- (6-3.south east) ;
\end{NiceArray}$
```

Typst

```
$ nicearray(column-gap: #0.28em,
  row-gap: #((0.2em,) * 5 + (0.2em + 3pt,)),
  dots: #(symbols: false),
  foreground: #(g => for i in range(5) {
    let (a, b) = (g.cells.at(i).at(0),
                  g.cells.at(i + 1).at(2))
    place(line(start: (a.x, a.y),
      end: (b.x + b.width, b.y + b.height),
      stroke: (paint: red.transparentize(60%),
               thickness: 3pt)))
  }),
  u_1, -, u_0, =, r;
  u_2, -, u_1, =, r;
  u_3, -, u_2, =, r;
  u_4, -, u_3, =, r;
  #hide($u_5$), , #hide($u_4$), dots.v, ;
  u_n, -, u_(n-1), =, r;
  hline();
  u_n, -, u_0, =, n r) $
```

$$\begin{array}{ccccc}
u_1 & - & u_0 & = & r \\
u_2 & - & u_1 & = & r \\
u_3 & - & u_2 & = & r \\
u_4 & - & u_3 & = & r \\
 & & \vdots & & \\
u_n & - & u_{n-1} & = & r \\
\hline
u_n & - & u_0 & = & nr
\end{array}$$

The “medium nodes” are the tiles of the cells, which reach the middle of the gaps.

LaTeX — nicematrix manual, p. 67

```
$\begin{NiceMatrix}
\CodeBefore
  \tikz \draw [fill=red!15] (7-|4) |- (8-|5) |-
(9-|6) |- cycle ;
\Body
  1 \\
  1 & 1 \\
  1 & 2 &  1 \\
  1 & 3 &  3 &  1 \\
  1 & 4 &  6 &  4 &  1 \\
  1 & 5 & 10 & 10 &  5 &  1 \\
  1 & 6 & 15 & 20 & 15 &  6 &  1 \\
  1 & 7 & 21 & 35 & 35 & 21 &  7 & 1 \\
  1 & 8 & 28 & 56 & 70 & 56 & 28 & 8 & 1
\end{NiceMatrix}$
```

Typst

```
$ nicearray(background: #(g => {
    let y(k) = g.rows.at(k).tile-top
    let x(k) = g.cols.at(k).tile-left
    place(polygon(fill: red.lighten(85%),
      stroke: 0.5pt,
      (x(3), y(6)), (x(3), y(7)), (x(4), y(7)),
      (x(4), y(8)), (x(5), y(8)), (x(5), y(6))))
  }),
  1;
  1, 1;
  1, 2, 1;
  1, 3, 3, 1;
  1, 4, 6, 4, 1;
  1, 5, 10, 10, 5, 1;
  1, 6, 15, 20, 15, 6, 1;
  1, 7, 21, 35, 35, 21, 7, 1;
  1, 8, 28, 56, 70, 56, 28, 8, 1) $
```

$$\begin{array}{ccccccccc}
1 & & & & & & & & \\
1 & 1 & & & & & & & \\
1 & 2 & 1 & & & & & & \\
1 & 3 & 3 & 1 & & & & & \\
1 & 4 & 6 & 4 & 1 & & & & \\
1 & 5 & 10 & 10 & 5 & 1 & & & \\
1 & 6 & 15 & 20 & 15 & 6 & 1 & & \\
1 & 7 & 21 & 35 & 35 & 21 & 7 & 1 & \\
1 & 8 & 28 & 56 & 70 & 56 & 28 & 8 & 1
\end{array}$$

The nodes `i-|j` of the rules are the edges of the tiles (`tile-top`, `tile-left`, ...).

LaTeX — nicematrix manual, p. 68

```
$\begin{pNiceArray}{ccc|c}
  2 & 1 & 3 & 0 \\
  3 & 3 & 1 & 0 \\
  3 & 3 & 1 & 0
\CodeAfter
  \tikz \draw [red] (3.4-|1) -- (3.4-|last) ;
\end{pNiceArray}$
```

Typst

```
$ nicemat(vlines: 3, foreground: #(g => {
    let r = g.rows.at(2)
    let y = r.tile-top + 0.4 * (r.tile-bottom -
r.tile-top)
    place(line(start: (g.main.x, y),
      end: (g.main.x + g.main.width, y), stroke:
red))
  }),
  2, 1, 3, 0;
  3, 3, 1, 0;
  3, 3, 1, 0) $
```

$$\begin{pmatrix} 2 & 1 & 3 & 0 \\ 3 & 3 & 1 & 0 \\ 3 & 3 & 1 & 0 \end{pmatrix}$$

13 Technical remarks

LaTeX — nicematrix manual, p. 70

```
$A = \begin{pNiceMatrix}
  1      & \Cdots &        & 1      \\
  a+b    & \Ddots &        & \Vdots \\
  \Vdots & \Ddots &        &        \\
  a+b    & \Cdots & a+b    & 1
\end{pNiceMatrix}$
```

Typst

```
$ A = nicemat(
  1, dots.c, , 1;
  a+b, ddots(fill: #red), , dots.v;
  dots.v, dots.down, , ;
  a+b, dots.c, a+b, 1) $
```

$$A = \begin{pmatrix} 1 & \cdots & \cdots & 1 \\ a+b & \ddots & & \vdots \\ \vdots & & \ddots & \vdots \\ a+b & \cdots & a+b & 1 \end{pmatrix}$$

The first diagonal leader (here in red) gives the direction of the others.

LaTeX — nicematrix manual, p. 70

```
$A = \begin{pNiceMatrix}
  1      & \Cdots &        & 1      \\
  a+b    &        &        & \Vdots \\
  \Vdots & \Ddots & \Ddots &        \\
  a+b    & \Cdots & a+b    & 1
\end{pNiceMatrix}$
```

Typst

```
$ A = nicemat(
  1, dots.c, , 1;
  a+b, , , dots.v;
  dots.v, ddots(fill: #red), dots.down, ;
  a+b, dots.c, a+b, 1) $
```

$$A = \begin{pmatrix} 1 & \cdots & \cdots & 1 \\ a+b & \ddots & & \vdots \\ \vdots & & \ddots & \vdots \\ a+b & \cdots & a+b & 1 \end{pmatrix}$$

Same algorithm as nicematrix, but Typst's tighter columns make the first diagonal (in red) steeper, so the line parallel to it runs past the matrix; `dots: (parallel: false)` or wider columns avoid that.

14 Examples

LaTeX — nicematrix manual, p. 75

```
\color{blue}
\[\begin{NiceArray}
  [columns-width=1cm,hvlines,rules/color=black]
  {>{\color{black}}cccccc}
  f(x)   & \Block[C]{1-5}{+} \\
  f'(x)  & \Block[C]{1-2}{+} && \Block[C]{1-2}{+}
&& + \\
  f''(x) & - & \Block[C]{1-2}{-} && \Block[C]{1-2}
{+} \\
\end{NiceArray}\]
```

Typst

```
#text(fill: blue, $ nicearray(
  column-width: #1cm, stroke: #black,
  hlines: #true, vlines: #true,
  map-cells: #((i, j, it) =>
    if j == 0 { text(black, it) } else { it }),
  f(x), cell(+, colspan: #5);
  f'(x), cell(+, colspan: #2),
    cell(+, colspan: #2), +;
  f''(x), -, cell(-, colspan: #2),
    cell(+, colspan: #2)) $)
```

$f(x)$	+		
$f'(x)$	+	+	+
$f''(x)$	−	−	+

A sign chart.

LaTeX — nicematrix manual, p. 76

```
$\begin{NiceArray}{r|cw{c}{1cm}cw{c}{1cm}c}[cell-
space-limits=2pt]
x     & 0 & - & \frac{1}{e} & +  & +\infty \\
\Hline
f'(x) &   & - & 0 & + & \\
\Hline
      & 1 &   &   &   & +\infty \\
f     &   &   &   &   &         \\
      &   &   & (1/e)^{1/e} \\
\CodeAfter
  \begin{tikzpicture}
  \draw (3-|4.5) -- (5-|4.5) ;
  \draw ([xshift=3pt]2-|2) -- ([xshift=3pt]3-|2) ;
  \draw [->,red,shorten < = 3pt] (3-2) -- (5-4) ;
  \draw [->,red,shorten < = 3pt] (5-4) --
(3-6) ;
  \end{tikzpicture}
\end{NiceArray}$
```

Typst

```
$ nicearray(cell-space: #2pt, vlines: 1,
  align: #(right, center),
  column-width: #(auto, auto, 1cm, auto, 1cm),
  foreground: #(g => {
    let y(k) = g.rows.at(k).tile-top
    let m = g.cols.at(3)
    let x = (m.tile-left + m.tile-right) / 2
    place(line(start: (x, y(2)), end: (x, y(4))))
    let x = g.cols.at(1).tile-left + 3pt
    place(line(start: (x, y(1)), end: (x, y(2))))
  }),
  x, 0, -, 1/e, +, +oo;
  hline();
  f'(x), , -, 0, +, ;
  hline();
  , 1, , , , +oo;
  f, , , , , ;
  , , , (1\/e)^(1\/e),
  connect((2, 1), (4, 3), stroke: #red,
    shorten-start: #3pt),
  connect((4, 3), (2, 5), stroke: #red,
    shorten-start: #3pt)) $
```

x	0	−	$\frac{1}{e}$	+	$+\infty$
$f'(x)$	‖	−	0	+	
f	1	↘	$(1/e)^{1/e}$	↗	$+\infty$

A variation table: the arrows are `connect`s; `\/` is a slash that doesn't make a fraction.

LaTeX — nicematrix manual, p. 77

```
$\begin{pNiceMatrix}
  \Block[borders={bottom,right,tikz=dotted}]{2-2}{}
  1 & 2 & 0  & 0 & 0 & 0 \\
  4 & 5 & 0  & 0 & 0 & 0 \\
  0 & 0 & \Block[borders={all,tikz=dotted}]{2-2}{}
          7  & 1 & 0 & 0 \\
  0 & 0 & -1 & 2 & 0 & 0 \\
  0 & 0 & 0  & 0 &
\Block[borders={left,top,tikz=dotted}]{2-2}{}
                  3 & 4 \\
  0 & 0 & 0  & 0 & 1 & 4
\end{pNiceMatrix}$
```

Typst

```
#let dot = (dash: "dotted")
$ nicemat(
  1, 2, 0, 0, 0, 0;
  4, 5, 0, 0, 0, 0;
  0, 0, 7, 1, 0, 0;
  0, 0, -1, 2, 0, 0;
  0, 0, 0, 0, 3, 4;
  0, 0, 0, 0, 1, 4,
  region((0, 0), (1, 1),
    stroke: #(bottom: dot, right: dot)),
  region((2, 2), (3, 3), stroke: #dot),
  region((4, 4), (5, 5),
    stroke: #(left: dot, top: dot))) $
```

$$\begin{pmatrix} 1 & 2 & 0 & 0 & 0 & 0 \\ 4 & 5 & 0 & 0 & 0 & 0 \\ 0 & 0 & 7 & 1 & 0 & 0 \\ 0 & 0 & -1 & 2 & 0 & 0 \\ 0 & 0 & 0 & 0 & 3 & 4 \\ 0 & 0 & 0 & 0 & 1 & 4 \end{pmatrix}$$

The key `borders` of `\Block` is a stroke per side, as for `rect`.

LaTeX — nicematrix manual, p. 77

```
\setlength{\extrarowheight}{1mm}
\[\begin{vNiceArray}{cccc;ccc}[columns-width=6mm,
xdots/shorten += 3pt]
  a_0   &      &&       &b_0    &      &     \\
  a_1   &\Ddots&&       &b_1    &\Ddots&     \\
  \Vdots&\Ddots&&       &\Vdots &\Ddots&b_0  \\
  a_p   &      &&a_0    &       &      &b_1   \\
        &\Ddots&&a_1    &b_q    &      &\Vdots\\
        &      &&\Vdots &       &\Ddots&      \\
        &      &&a_p    &       &      &b_q
\end{vNiceArray}\]
```

Typst

```
$ nicemat(delim: "|", column-width: #6mm,
  cell-space: #(top: 1mm),
  dots: #(shorten: 0.3em + 3pt),
  a_0, , , , vline(stroke: #(dash: "dashed")),
    b_0, , ;
  a_1, dots.down, , , b_1, dots.down, ;
  dots.v, dots.down, , , dots.v, dots.down, b_0;
  a_p, , , a_0, , , b_1;
  , dots.down, , a_1, b_q, , dots.v;
  , , , dots.v, , dots.down, ;
  , , , a_p, , , b_q) $
```

$$\left|\begin{array}{cccc:ccc} a_0 & & & & b_0 & & \\ a_1 & \ddots & & & b_1 & \ddots & \\ \vdots & \ddots & \ddots & & \vdots & \ddots & b_0 \\ a_p & & a_0 & & b_q & & b_1 \\ & \ddots & a_1 & & & \ddots & \vdots \\ & & \vdots & & & & b_q \\ & & a_p & & & & \end{array}\right|$$

The resultant of two polynomials.

LaTeX — nicematrix manual, p. 78

```
\NiceMatrixOptions{code-for-last-col=\scriptstyle}
$\begin{pNiceMatrix}[vlines=-1,xdots/nullify,last-
col]
  1      & 1 & 1 &\Cdots &   & 1      & 0      & \\
  0      & 1 & 0 &\Cdots &   & 0      &        &
L_2 \gets L_2-L_1 \\
  0      & 0 & 1 &\Ddots &   & \Vdots &        &
L_3 \gets L_3-L_1 \\
         &   &   &\Ddots &   &        & \Vdots &
\Vdots \\
  \Vdots &   &   &\Ddots &   & 0      & \\
  0      &   &   &\Cdots & 0 & 1      & 0      &
L_n \gets L_n-L_1
\end{pNiceMatrix}$
```

Typst

```
$ nicemat(vlines: -1, last-col: #true,
  dots: #(nullify: true),
  map-cells: #((i, j, it) =>
    if j == 7 { $script(#it)$ } else { it }),
  1, 1, 1, dots.c, , 1, 0, ;
  0, 1, 0, dots.c, , 0, , L_2 <- L_2 - L_1;
  0, 0, 1, dots.down, , dots.v, , L_3 <- L_3 - L_1;
  , , , dots.down, , , dots.v, dots.v;
  dots.v, , , dots.down, , 0, ;
  0, , , dots.c, 0, 1, 0, L_n <- L_n - L_1) $
```

$$\left(\begin{array}{ccccccc|c} 1 & 1 & 1 & \cdots & & 1 & 0 & \\ 0 & 1 & 0 & \cdots & & 0 & \vdots & L_2 \leftarrow L_2 - L_1 \\ 0 & 0 & 1 & \ddots & & \vdots & \vdots & L_3 \leftarrow L_3 - L_1 \\ \vdots & & \ddots & \ddots & \ddots & \vdots & \vdots & \\ \vdots & & & \ddots & \ddots & 0 & \vdots & \\ 0 & \cdots & \cdots & \cdots & 0 & 1 & 0 & L_n \leftarrow L_n - L_1 \end{array}\right)$$

A linear system. `vlines: -1` is the last rule inside the matrix.

LaTeX — nicematrix manual, p. 78

```
\setcounter{MaxMatrixCols}{12}
\newcommand{\blue}{\color{blue}}
\[\begin{pNiceMatrix}%
    [last-row,last-col,xdots={nullify,line-
style={dashed,dash expand off,blue}}]
  1& & & \Vdots & & & & \Vdots & \\
  & \Ddots[line-style=standard] & \\
  & & 1 & \\
  \Cdots & & & \blue 0 & \Cdots & & & \blue 1 & & &
\Cdots & \blue \leftarrow i \\
  & & & & 1 & \\
  & & &\Vdots & & \Ddots[line-style=standard] & &
\Vdots & \\
  & & & & & & 1 & \\
  \Cdots & & & \blue 1 & \Cdots & & & \Cdots & \blue
0 & & & \Cdots & \blue \leftarrow j \\
  & & & & & & & & 1 & \\
  & & & & & & & & & \Ddots[line-style=standard] & \\
  & & & \Vdots & & & & \Vdots & & & & 1 & \\
  & & & \blue \overset{\uparrow}{i} & & & & \blue
\overset{\uparrow}{j} & \\
\end{pNiceMatrix}\]
```

Typst

```
#let bl(x) = text(fill: blue, x)
#set text(size: 9pt)
$ nicemat(last-row: #true, last-col: #true,
  dots: #(nullify: true,
    stroke: (paint: blue, dash: "dashed")),
  1, , , dots.v, , , , dots.v;
  , ddots(stroke: #auto);
  , , 1;
  dots.c, , , bl(0), dots.c, , , bl(1), , ,
    dots.c, bl(<- i);
  , , , , 1;
  , , , dots.v, , ddots(stroke: #auto), , dots.v;
  , , , , , , 1;
  dots.c, , , bl(1), dots.c, , dots.c, bl(0), , ,
    dots.c, bl(<- j);
  , , , , , , , , 1;
  , , , , , , , , , ddots(stroke: #auto);
  , , , dots.v, , , , dots.v, , , 1;
  , , , bl(limits(i)^arrow.t), , , ,
    bl(limits(j)^arrow.t)) $
```

$$\left(\begin{array}{ccccccccc} 1 & & & & & & & & \\ & \ddots & & & & & & & \\ & & 1 & & & & & & \\ \hdashline & & & 0 & & & 1 & & \\ \hdashline & & & & 1 & & & & \\ & & & & & \ddots & & & \\ & & & & & & 1 & & \\ \hdashline & & & 1 & & & 0 & & \\ \hdashline & & & & & & & 1 & \\ & & & & & & & & \ddots \\ & & & & & & & & 1 \end{array}\right) \begin{array}{l} \\ \\ \\ \leftarrow i \\ \\ \\ \\ \leftarrow j \\ \\ \\ \end{array}$$
$$\begin{array}{ccccccccc} & & & \uparrow & & & \uparrow & & \\ & & & i & & & j & & \end{array}$$

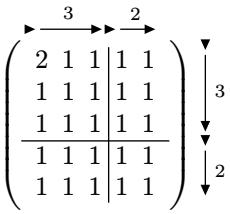
Leaders that are no longer dotted; `stroke: #auto` gives back the dots for one leader.

LaTeX — nicematrix manual, p. 79

```
{\NiceMatrixOptions{xdots={horizontal-labels,line-
style = <->}}}
$\begin{pNiceArray}{ccc|cc}[first-row,last-
col,margin]
  \Hdotsfor{3}^{3} & \Hdotsfor{2}^{2} \\
  2 & 1 & 1 & 1 & \Vdotsfor{3}^{3}\\
  1 & 1 & 1 & 1 & 1 \\
  1 & 1 & 1 & 1 & 1 \\
\Hline
  1 & 1 & 1 & 1 & \Vdotsfor{2}^{2} \\
  1 & 1 & 1 & 1 & 1 \\
\end{pNiceArray}$
```

Typst

```
$ nicemat(first-row: #true, last-col: #true,
  margin: #(x: 5pt), vlines: 3,
  dots: #(horizontal-labels: true,
    stroke: 0.5pt, marks: "<->"),
  cdots(span: #3, above: 3),
    cdots(span: #2, above: 2);
  2, 1, 1, 1, 1, vdots(span: #3, above: 3);
  1, 1, 1, 1, 1;
  1, 1, 1, 1, 1;
  hline();
  1, 1, 1, 1, 1, vdots(span: #2, above: 2);
  1, 1, 1, 1, 1) $
```



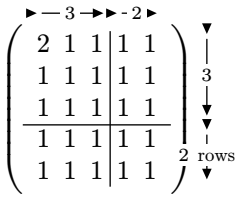
Illustrating the numbers of rows and columns: `line-style = <->` is a stroke with `marks`.

LaTeX — nicematrix manual, p. 79

```
{\NiceMatrixOptions{xdots={horizontal-labels,line-
style = <->}}}
$\begin{pNiceArray}{ccc|cc}[first-row,last-
col,margin]
  \Hdotsfor{3}:{3} & \Hdotsfor{2}:{2} \\
  2 & 1 & 1 & 1 & \Vdotsfor{3}:{3} \\
  1 & 1 & 1 & 1 & 1 \\
  1 & 1 & 1 & 1 & 1 \\
\Hline
  1 & 1 & 1 & 1 &
  \Vdotsfor{2}:{2 \rlap{ \smash{rows}}} \\
  1 & 1 & 1 & 1 & 1 \\
\end{pNiceArray}$
```

Typst

```
$ nicemat(first-row: #true, last-col: #true,
  margin: #(x: 5pt), vlines: 3,
  dots: #(horizontal-labels: true,
    stroke: 0.5pt, marks: "<->"),
  cdots(span: #3, middle: 3),
    cdots(span: #2, middle: 2);
  2, 1, 1, 1, 1, vdots(span: #3, middle: 3);
  1, 1, 1, 1, 1;
  1, 1, 1, 1, 1;
  hline();
  1, 1, 1, 1, 1,
    vdots(span: #2, middle: 2 "rows");
  1, 1, 1, 1, 1) $
```



LaTeX — nicematrix manual, p. 79

```
\NiceMatrixOptions{xdots/horizontal-labels}
$\begin{pNiceArray}{ccc|cc}[first-row,last-
col,margin]
  \Hbrace{3}{3} & \Hbrace{2}{2} \\
  2 & 1 & 1 & 1 & 1 & \Vbrace{3}{3} \\
  1 & 1 & 1 & 1 & 1 \\
  1 & 1 & 1 & 1 & 1 \\
\Hline
  1 & 1 & 1 & 1 & 1 & \Vbrace{2}{2} \\
  1 & 1 & 1 & 1 & 1 \\
\end{pNiceArray}$
```

Typst

```
$ nicemat(first-row: #true, last-col: #true,
  margin: #(x: 5pt), vlines: 3,
  hbrace(3, span: #3), hbrace(2, span: #2);
  2, 1, 1, 1, 1, vbrace(3, span: #3);
  1, 1, 1, 1, 1;
  1, 1, 1, 1, 1;
  hline();
  1, 1, 1, 1, 1, vbrace(2, span: #2);
  1, 1, 1, 1, 1) $
```

$$\overbrace{\phantom{\begin{matrix}2&1&1\\1&1&1\\1&1&1\end{matrix}}}^{3}\ \overbrace{\phantom{\begin{matrix}1&1\\1&1\\1&1\end{matrix}}}^{2}$$
$$\left(\begin{array}{ccc|cc}2&1&1&1&1\\1&1&1&1&1\\1&1&1&1&1\\\hline1&1&1&1&1\\1&1&1&1&1\end{array}\right)\begin{matrix}\}3\\\}2\end{matrix}$$

LaTeX — nicematrix manual, p. 80

```
$\begin{pNiceArray}{ccc|cc}[margin,last-col]
  2 & 1 & 1 & 1 & \Block{3-1}{\quad 3} \\
  1 & 1 & 1 & 1 & 1 \\
  1 & 1 & 1 & 1 & 1 \\
\Hline
  1 & 1 & 1 & 1 & \Block{2-1}{\quad 2} \\
  1 & 1 & 1 & 1 & 1 \\
\CodeAfter
  \OverBrace[shorten,yshift=1.5mm]{1-1}{1-3}{3}
  \OverBrace[shorten,yshift=1.5mm]{1-4}{1-5}{2}
  \SubMatrix{.}{1-1}{3-5}{\rbrace}[xshift=3.5mm]
  \SubMatrix{.}{4-1}{5-5}{\rbrace}[xshift=3.5mm]
\end{pNiceArray}$
```

Typst

```
$ nicemat(last-col: #true, margin: #(x: 5pt),
  vlines: 3,
  2, 1, 1, 1, 1, cell(quad 3, rowspan: #3);
  1, 1, 1, 1, 1;
  1, 1, 1, 1, 1;
  hline();
  1, 1, 1, 1, 1, cell(quad 2, rowspan: #2);
  1, 1, 1, 1, 1,
  hbrace((0, 0), (0, 2), 3,
    shorten: #true, shift: #1.5mm),
  hbrace((0, 3), (0, 4), 2,
    shorten: #true, shift: #1.5mm),
  submatrix((0, 0), (2, 4), delim: #(none, "}"),
    xshift: #3.5mm, reserve: #false),
  submatrix((3, 0), (4, 4), delim: #(none, "}"),
    xshift: #3.5mm, reserve: #false)) $
```

$$\overbrace{\phantom{\begin{matrix}2&1&1\\1&1&1\\1&1&1\end{matrix}}}^{3}\ \overbrace{\phantom{\begin{matrix}1&1\\1&1\\1&1\end{matrix}}}^{2}$$
$$\left(\begin{array}{ccc|cc}2&1&1&1&1\\1&1&1&1&1\\1&1&1&1&1\\\hline1&1&1&1&1\\1&1&1&1&1\end{array}\right)\begin{matrix}\}3\\\}2\end{matrix}$$

`\SubMatrix{.}{..}{..}{\rbrace}` is a sub-matrix with one delimiter. As in the manual, the result is disappointing: `hbrace` / `vbrace` are better for this.

LaTeX — nicematrix manual, p. 81

```
\begin{NiceMatrixBlock}[auto-columns-width]
\NiceMatrixOptions
  {
    light-syntax,
    last-col, code-for-last-col =
\color{blue}\scriptstyle,
    vlines = 5 ,
    matrix/columns-type = r ,
    no-cell-nodes % only for speedup
  }
\setlength{\extrarowheight}{1mm}

\quad $\begin{pNiceMatrix}
12  -8  7  5   3 {} ;
 3 -18 12  1   4    ;
-3 -46 29 -2 -15    ;
 9  10 -5  4   7
\end{pNiceMatrix}$

\smallskip
\quad $\begin{pNiceMatrix}
12  -8   7  5   3 ;
0   64 -41  1  19  { L_2 \gets L_1-4L_2  } ;
0 -192 123 -3 -57  { L_3 \gets L_1+4L_3  } ;
0 -64   41 -1 -19  { L_4 \gets 3L_1-4L_4 } ;
\end{pNiceMatrix}$

\smallskip
\quad $\begin{pNiceMatrix}
12 -8   7 5  3 ;
0  64 -41 1 19 ;
0   0   0 0  0 { L_3 \gets 3 L_2 + L_3 }
\end{pNiceMatrix}$

\smallskip
\quad $\begin{pNiceMatrix}
12 -8   7 5  3 {} ;
0  64 -41 1 19    ;
\end{pNiceMatrix}$
\end{NiceMatrixBlock}
```

Typst

```
#nicemat-group[
  #let st = nicemat.with(align: right,
    last-col: true, vlines: 4,
    cell-space: (top: 1mm),
    map-cells: (i, j, it) => if j == 5 {
      text(blue, $script(it)$) } else { it })
  $ st(12, -8, 7, 5, 3, ;
    3, -18, 12, 1, 4, ;
    -3, -46, 29, -2, -15, ;
    9, 10, -5, 4, 7, ) $
  $ st(12, -8, 7, 5, 3, ;
    0, 64, -41, 1, 19, L_2 <- L_1 - 4 L_2;
    0, -192, 123, -3, -57, L_3 <- L_1 + 4 L_3;
    0, -64, 41, -1, -19, L_4 <- 3 L_1 - 4 L_4) $
  $ st(12, -8, 7, 5, 3, ;
    0, 64, -41, 1, 19, ;
    0, 0, 0, 0, 0, L_3 <- 3 L_2 + L_3) $
  $ st(12, -8, 7, 5, 3, ;
    0, 64, -41, 1, 19, ) $
]
```

$$\left(\begin{array}{rrrr|r} 12 & -8 & 7 & 5 & 3 \\ 3 & -18 & 12 & 1 & 4 \\ -3 & -46 & 29 & -2 & -15 \\ 9 & 10 & -5 & 4 & 7 \end{array}\right)$$

$$\left(\begin{array}{rrrr|r} 12 & -8 & 7 & 5 & 3 \\ 0 & 64 & -41 & 1 & 19 \\ 0 & -192 & 123 & -3 & -57 \\ 0 & -64 & 41 & -1 & -19 \end{array}\right) \begin{array}{l} \\ L_2 \leftarrow L_1 - 4L_2 \\ L_3 \leftarrow L_1 + 4L_3 \\ L_4 \leftarrow 3L_1 - 4L_4 \end{array}$$

$$\left(\begin{array}{rrrr|r} 12 & -8 & 7 & 5 & 3 \\ 0 & 64 & -41 & 1 & 19 \\ 0 & 0 & 0 & 0 & 0 \end{array}\right) \quad L_3 \leftarrow 3L_2 + L_3$$

$$\left(\begin{array}{rrrr|r} 12 & -8 & 7 & 5 & 3 \\ 0 & 64 & -41 & 1 & 19 \end{array}\right)$$

Stacks of matrices. `nicemat-group` shares the widths of the columns and of the delimiters, so `delimiters/max-width` (p. 83) is not needed.

LaTeX — nicematrix manual, p. 82

```latex
\setlength{\extrarowheight}{1mm}
\[\begin{NiceMatrix}[r, last-col=6, code-for-last-
col = \scriptstyle \color{blue}]
12 &  -8  &  7 & 5 &  3 \\
 3 & -18  & 12 & 1 &  4 \\
-3 & -46  & 29 &-2 &-15 \\
 9 & 10   &-5  &4  & 7 \\[1mm]
12 & -8   & 7  &5  & 3 \\
0  & 64   &-41 & 1 & 19 & L_2 \gets L_1-4L_2  \\
0  & -192 &123 &-3 &-57 & L_3 \gets L_1+4L_3  \\
0  & -64  & 41 &-1 &-19 & L_4 \gets 3L_1-4L_4 \
\[1mm]
12 & -8   &7   &5  & 3 \\
0  & 64   &-41 &1  &19 \\
0  &  0   &0   &0  & 0  & L_3 \gets 3L_2+L_3 \
\[1mm]
12 & -8   &7   &5  & 3 \\
0  & 64   &-41 & 1 & 19 \\
\CodeAfter [sub-matrix/vlines=4]
   \SubMatrix({1-1}{4-5})
   \SubMatrix({5-1}{8-5})
   \SubMatrix({9-1}{11-5})
   \SubMatrix({12-1}{13-5})
\end{NiceMatrix}\]
```

Typst

```typst
#set text(size: 9.5pt)
$ nicemat(delim: #none, align: #right,
  last-col: #true, cell-space: #(top: 1mm),
  row-gap: #range(12).map(k =>
    if k in (3, 7, 10) { 0.2em + 1mm } else
{ 0.2em }),
  map-cells: #((i, j, it) => if j == 5 {
    text(blue, $script(#it)$) } else { it }),
  12, -8, 7, 5, 3, ;
  3, -18, 12, 1, 4, ;
  -3, -46, 29, -2, -15, ;
  9, 10, -5, 4, 7, ;
  12, -8, 7, 5, 3, ;
  0, 64, -41, 1, 19, L_2 <- L_1 - 4 L_2;
  0, -192, 123, -3, -57, L_3 <- L_1 + 4 L_3;
  0, -64, 41, -1, -19, L_4 <- 3 L_1 - 4 L_4;
  12, -8, 7, 5, 3, ;
  0, 64, -41, 1, 19, ;
  0, 0, 0, 0, 0, L_3 <- 3 L_2 + L_3;
  12, -8, 7, 5, 3, ;
  0, 64, -41, 1, 19, ,
  submatrix((0, 0), (3, 4), vlines: 4),
  submatrix((4, 0), (7, 4), vlines: 4),
  submatrix((8, 0), (10, 4), vlines: 4),
  submatrix((11, 0), (12, 4), vlines: 4)) $
```

$$
\left(\begin{array}{rrrr|r}
12 & -8 & 7 & 5 & 3 \\
3 & -18 & 12 & 1 & 4 \\
-3 & -46 & 29 & -2 & -15 \\
9 & 10 & -5 & 4 & 7
\end{array}\right)
\left(\begin{array}{rrrr|r}
12 & -8 & 7 & 5 & 3 \\
0 & 64 & -41 & 1 & 19 \\
0 & -192 & 123 & -3 & -57 \\
0 & -64 & 41 & -1 & -19
\end{array}\right)
\begin{array}{l}
\\ L_2 \leftarrow L_1 - 4L_2 \\ L_3 \leftarrow L_1 + 4L_3 \\ L_4 \leftarrow 3L_1 - 4L_4
\end{array}
\left(\begin{array}{rrrr|r}
12 & -8 & 7 & 5 & 3 \\
0 & 64 & -41 & 1 & 19 \\
0 & 0 & 0 & 0 & 0
\end{array}\right)
\begin{array}{l}
\\ \\ L_3 \leftarrow 3L_2 + L_3
\end{array}
\left(\begin{array}{rrrr|r}
12 & -8 & 7 & 5 & 3 \\
0 & 64 & -41 & 1 & 19
\end{array}\right)
$$

One matrix with four sub-matrices; `sub-matrix/vlines` and `vlines-in-sub-matrix` (p. 84) are the `vlines` of each sub-matrix.

LaTeX — nicematrix manual, p. 84

```
$\begin{pNiceArray}{>{\strut}cccc}[margin,rules/
color=blue,no-cell-nodes]
\Block[draw]{}{a_{11}} & a_{12} & a_{13} & a_{14} \
\
a_{21} & \Block[draw]{}{a_{22}} & a_{23} & a_{24} \
\
a_{31} & a_{32} & \Block[draw]{}{a_{33}} & a_{34} \
\
a_{41} & a_{42} & a_{43} & \Block[draw]{}{a_{44}} \
\
\end{pNiceArray}$
```

Typst

```
$ nicemat(margin: #(x: 5pt), stroke: #blue,
  cell(a_11, stroke: #true), a_12, a_13, a_14;
  a_21, cell(a_22, stroke: #true), a_23, a_24;
  a_31, a_32, cell(a_33, stroke: #true), a_34;
  a_41, a_42, a_43, cell(a_44, stroke: #true)) $
```

$$\begin{pmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{pmatrix}$$

How to highlight cells. `stroke: #true` uses the stroke of the rules.

LaTeX — nicematrix manual, p. 84

```
$\begin{pNiceArray}{>{\strut}cccc}% <-- % mandatory
   [margin, extra-margin=2pt,no-cell-nodes]
  \rowcolor{red!15}A_{11} & A_{12} & A_{13} &
A_{14} \\
  A_{21} & \rowcolor{red!15}A_{22} & A_{23} &
A_{24} \\
  A_{31} & A_{32} & \rowcolor{red!15}A_{33} &
A_{34} \\
  A_{41} & A_{42} & A_{43} & \rowcolor{red!15}
A_{44}
\end{pNiceArray}$
```

Typst

```
$ nicemat(margin: #(x: 7pt),
  fill: #((i, j) => if j >= i
{ red.lighten(85%) }),
  A_11, A_12, A_13, A_14;
  A_21, A_22, A_23, A_24;
  A_31, A_32, A_33, A_34;
  A_41, A_42, A_43, A_44) $
```

$$\begin{pmatrix} A_{11} & A_{12} & A_{13} & A_{14} \\ A_{21} & A_{22} & A_{23} & A_{24} \\ A_{31} & A_{32} & A_{33} & A_{34} \\ A_{41} & A_{42} & A_{43} & A_{44} \end{pmatrix}$$

LaTeX — nicematrix manual, p. 85

```
\NewDocumentCommand{\OneRow}{}
{\Block[tikz={xoffset=2.5pt,yoffset=0.8pt,fill=red!
15,rounded corners = 2pt}]{1-*}{}}
$\begin{pNiceArray}{ccc}[last-col, margin = 2pt]
\OneRow a & a + b & a + b + c & L_1 \\
\OneRow a & a     & a + b     & L_2 \\
\OneRow a & a     & a         & L_3
\end{pNiceArray}$
```

Typst

```
#let onerow(i) = region(row: i,
  outset: (x: -2.5pt, y: -0.8pt),
  fill: red.lighten(85%), radius: 2pt)
$ nicemat(last-col: #true, margin: #(x: 2pt),
  a, a + b, a + b + c, L_1;
  a, a, a + b, L_2;
  a, a, a, L_3,
  onerow(0), onerow(1), onerow(2)) $
```

$$\begin{pmatrix} a & a+b & a+b+c \\ a & a & a+b \\ a & a & a \end{pmatrix} \begin{matrix} L_1 \\ L_2 \\ L_3 \end{matrix}$$

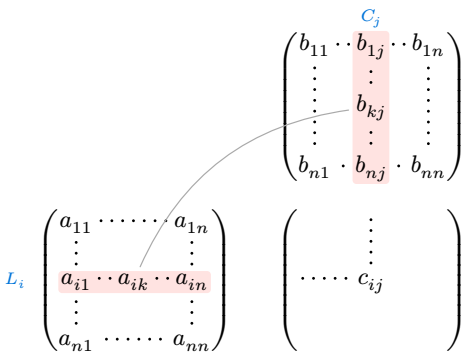
A block with TikZ offsets is a `region` with a negative `outset`; `row: i` covers the row inside the delimiters.

LaTeX — nicematrix manual, p. 85

```
\[\begin{NiceArray}{*{6}{c}@{\hspace{6mm}}*{5}{c}}
[xdots/nullify]
\CodeBefore [create-cell-nodes]
  \SubMatrix({2-7}{6-last})
  \SubMatrix({7-2}{last-6})
  \SubMatrix({7-7}{last-last})
  \begin{tikzpicture}
    \node [highlight = (9-2) (9-6)] { } ;
    \node [highlight = (2-9) (6-9)] { } ;
  \end{tikzpicture}
\Body
    &        &        &        &        &
&        & \color{blue}\scriptstyle C_j \\
    &        &        &        &        &
b_{11} & \Cdots & b_{1j} & \Cdots & b_{1n} \\
    &        &        &        &        &
\Vdots &        & \Vdots &        & \Vdots \\
    &        &        &        &        &
&        & b_{kj} \\
    &        &        &        &        &
&        & \Vdots \\
    &        &        &        &        &
b_{n1} & \Cdots & b_{nj} & \Cdots & b_{nn} \\[3mm]
    & a_{11} & \Cdots &        &        & a_{1n} \\
    & \Vdots &        &        &        & \Vdots &
&        & \Vdots \\
\color{blue}\scriptstyle L_i
    & a_{i1} & \Cdots & a_{ik} & \Cdots & a_{in} &
\Cdots &        & c_{ij} \\
    & \Vdots &        &        &        & \Vdots \\
    & a_{n1} & \Cdots &        &        & a_{nn}  \
\
\CodeAfter
\tikz \draw [gray,shorten > = 1mm, shorten < = 1mm]
(9-4.north) to [bend left] (4-9.west) ;
\end{NiceArray}\]
```

Typst

```
#set text(size: 9pt)
#let hl(from, to) = region(from, to, fit:
"content",
  outset: 1pt, radius: 0.5mm, fill:
red.lighten(85%))
$ nicearray(dots: #(nullify: true),
  column-gap: #range(10).map(k =>
    if k == 5 { 6mm } else { 0.5em }),
  row-gap: #range(10).map(k =>
    if k == 5 { 0.2em + 3mm } else { 0.2em }),
  map-cells: #((i, j, it) => if (i, j) in ((0, 8),
(8, 0)) {
    text(blue, $script(#it)$) } else { it }),
  , , , , , , , , C_j;
  , , , , , , b_11, dots.c, b_(1 j), dots.c, b_(1
n);
  , , , , , , dots.v, , dots.v, , dots.v;
  , , , , , , , , b_(k j);
  , , , , , , , , dots.v;
  , , , , , , b_(n 1), dots.c, b_(n j), dots.c,
b_(n n);
  , a_11, dots.c, , , a_(1 n);
  , dots.v, , , , dots.v, , , dots.v;
  L_i, a_(i 1), dots.c, a_(i k), dots.c, a_(i n),
    dots.c, , c_(i j);
  , dots.v, , , , dots.v;
  , a_(n 1), dots.c, , , a_(n n),
  submatrix((1, 6), (5, 10)),
  submatrix((6, 1), (10, 5)),
  submatrix((6, 6), (10, 10)),
  hl((8, 1), (8, 5)), hl((1, 8), (5, 8)),
  dotline((8, 3), (3, 8), bend: 30,
    stroke: #gray, shorten: #1mm)) $
```



The sub-matrices stop the leaders (like `\SubMatrix` in the `\CodeBefore`); the highlights are regions fitted to the contents; the curve is a bent `dotline ( to [bend left] )`.

15 Examples not ported

82 of the 177 examples of the manual are not ported:

line	page	beginning of the code	reason
L713	8	`\begin{NiceTabular}{cw{c}{2cm}w{c}{3cm}c}`	text table (`{NiceTabular}` , `{tabular}`)
L755	9	`\begin{NiceTabular}{@{}>{\bfseries}lr@{}}` `\hline`	text table (`{NiceTabular}` , `{tabular}`)
L837	10	`\begin{NiceTabular}{@{}c!{\qquad}ccc!` `{\qquad}ccc@{}}`	text table (`{NiceTabular}` , `{tabular}`)
L863	10	`\begin{NiceTabular}{@{}c!{\qquad}ccc!` `{\qquad}ccc@{}}`	text table (`{NiceTabular}` , `{tabular}`)
L898	11	`\begin{NiceTabular}[hvlines]{ccc}`	text table (`{NiceTabular}` , `{tabular}`)

line	page	beginning of the code	reason
L962	12	`\begin{NiceTabular}{ccc}`	text table (`{NiceTabular}` , `{tabular}`)
L975	12	`\begin{NiceTabular}{ccc}`	text table (`{NiceTabular}` , `{tabular}`)
L987	12	`\begin{NiceTabular}{ccc}`	text table (`{NiceTabular}` , `{tabular}`)
L1000	12	`\begin{NiceTabular}{ccc}`	text table (`{NiceTabular}` , `{tabular}`)
L1051	13	`\begin{NiceTabular}{ll}%`	text table (`{NiceTabular}` , `{tabular}`)
L1067	13	`\begin{NiceTabular}{lccccc}%`	text table (`{NiceTabular}` , `{tabular}`)
L1102	13	`\begin{NiceTabular}{lr}[hvlines,create-blocks-in-col=1]`	text table (`{NiceTabular}` , `{tabular}`)
L1145	14	`\begin{NiceTabular}{|c|c|} \hline`	text table (`{NiceTabular}` , `{tabular}`)
L1187	14	`\newcolumntype{I}{!{\vrule}}`	text table (`{NiceTabular}` , `{tabular}`)
L1221	15	`\setlength{\arrayrulewidth}{2pt}`	text table (`{NiceTabular}` , `{tabular}`)
L1236	15	`\setlength{\arrayrulewidth}{2pt}`	text table (`{NiceTabular}` , `{tabular}`)
L1275	15	`\begin{NiceTabular}{|ccc|}{[rules/color=[gray]{0.9},rul…`	text table (`{NiceTabular}` , `{tabular}`)
L1343	16	`\begin{NiceTabular}{ | c |[color=blue] c |}`	text table (`{NiceTabular}` , `{tabular}`)
L1419	17	`\begin{NiceTabular}{cccc}[hvlines,rules={color=blue,wid…`	text table (`{NiceTabular}` , `{tabular}`)
L1502	18	`\NiceMatrixOptions{cell-space-top-limit=3pt}`	text table (`{NiceTabular}` , `{tabular}`)
L1523	18	`\NiceMatrixOptions{cell-space-top-limit=3pt}`	text table (`{NiceTabular}` , `{tabular}`)
L1553	18	`\begin{NiceTabular}{cccc}[hvlines,corners=E]`	text table (`{NiceTabular}` , `{tabular}`)
L1702	20	`\begin{NiceTabular}{lcIcIc}[custom-line = {letter=I, co…`	text table (`{NiceTabular}` , `{tabular}`)
L1734	20	`\NiceMatrixOptions`	text table (`{NiceTabular}` , `{tabular}`)
L1940	22	`\NiceMatrixOptions`	text table (`{NiceTabular}` , `{tabular}`)
L1987	23	`\begin{NiceTabular}{cc|[color=blue,start=2]ccc}`	text table (`{NiceTabular}` , `{tabular}`)
L2022	23	`\NiceMatrixOptions`	TikZ decoration
L2057	23	`\begin{NiceTabular}{ccc}[vlines]`	text table (`{NiceTabular}` , `{tabular}`)
L2080	23	`\begin{NiceTabular}{ccc}[vlines,rules/fix-vertex]`	text table (`{NiceTabular}` , `{tabular}`)
L2121	24	`\NiceMatrixOptions`	TikZ decoration
L2152	24	`\renewcommand{\arraystretch}{1.3}`	text table (`{NiceTabular}` , `{tabular}`)
L2191	24	`\begin{NiceTabular}{cFcc}[hvlines]`	text table (`{NiceTabular}` , `{tabular}`)
L2283	25	`\begin{pNiceArray}{\textsl{preamble}}`	syntax summary
L2365	26	`\begin{NiceTabular}{ccc}[hvlines]`	text table (`{NiceTabular}` , `{tabular}`)
L2382	26	`\begin{NiceTabular}{ccc}[hvlines]`	text table (`{NiceTabular}` , `{tabular}`)
L2494	28	`\begin{NiceTabular}{clr}[hvlines]`	text table (`{NiceTabular}` , `{tabular}`)
L2517	28	`\begin{NiceTabular}{c}[rounded-corners]`	text table (`{NiceTabular}` , `{tabular}`)
L2546	28	`\begin{NiceTabular}{c}`	text table (`{NiceTabular}` , `{tabular}`)
L2571	29	`\begin{NiceTabular}{ccccccc}{[corners=E,margin,hvlines,…`	text table (`{NiceTabular}` , `{tabular}`)
L2602	29	`\begin{NiceTabular}{ccccccc}{[corners=E,margin,hvlines,…`	text table (`{NiceTabular}` , `{tabular}`)
L2626	30	`\begin{NiceTabular}[c]{lSSSS}[no-cell-nodes] {% no-cell…`	text table (`{NiceTabular}` , `{tabular}`)
L2667	30	`\begin{NiceTabular}{ccccc}[hvlines,no-cell-nodes] {% no…`	text table (`{NiceTabular}` , `{tabular}`)
L2726	31	`\NewDocumentCommand { \Blue } { } { \columncolor{blue!1…`	text table (`{NiceTabular}` , `{tabular}`)
L2752	31	`\begin{NiceTabular}{c}[hvlines]`	text table (`{NiceTabular}` , `{tabular}`)
L2770	32	`\begin{NiceTabular}{lr}[hvlines,create-blocks-in-col=1]`	text table (`{NiceTabular}` , `{tabular}`)
L2857	33	`\begin{NiceTabular}{cccc}`	text table (`{NiceTabular}` , `{tabular}`)
L2887	33	`\begin{NiceTabular}{W{c}{2cm}cc}[hvlines]`	text table (`{NiceTabular}` , `{tabular}`)
L2997	34	`\fbox{%`	text layout
L3008	34	`\fbox{%`	text layout
L3029	35	`\begin{NiceTabular}[corners=NW,hvlines]{V{3cm}V{3cm}V{3…`	text table (`{NiceTabular}` , `{tabular}`)
L3106	36	`\begin{NiceTabular}[width=9cm]{X[c,m]X[0.5,c,m]}[hvline…`	text table (`{NiceTabular}` , `{tabular}`)
L3857	44	`decoration = { brace , raise = -0.15 em } ,`	TikZ decoration
L4254	49	`\begin{NiceTabular}{@{}ll@{}}`	text table (`{NiceTabular}` , `{tabular}`)
L4436	51	`\renewcommand{\arraystretch}{1.3}`	text table (`{NiceTabular}` , `{tabular}`)
L4452	51	`\renewcommand{\arraystretch}{1.3}`	text table (`{NiceTabular}` , `{tabular}`)
L4481	51	`\renewcommand{\arraystretch}{1.4}`	text table (`{NiceTabular}` , `{tabular}`)
L4498	52	`\begin{NiceTabular}{ccccc}`	text table (`{NiceTabular}` , `{tabular}`)

line	page	beginning of the code	reason
L4612	53	`\begin{NiceTabular}{@{}llr@{}}`	text table (`{NiceTabular}` , `{tabular}`)
L4686	54	`\begin{table}[hbt]`	text table (`{NiceTabular}` , `{tabular}`)
L4942	57	`\renewcommand{\arraystretch}{1.3}`	text table (`{NiceTabular}` , `{tabular}`)
L4970	57	`\renewcommand{\arraystretch}{1.3}`	text table (`{NiceTabular}` , `{tabular}`)
L5026	58	`\begin{NiceTabular}`	text table (`{NiceTabular}` , `{tabular}`)
L5057	59	`\begin{NiceTabular}{ccc}[hvlines,cell-space-limits=3pt]`	text table (`{NiceTabular}` , `{tabular}`)
L5075	59	`\begin{NiceTabular}{ccc}[hvlines,cell-space-limits=3pt]`	text table (`{NiceTabular}` , `{tabular}`)
L5601	64	`\pgfset`	LaTeX configuration
L6071	68	`\DeclareKeys[nicematrix/environments] {arraystretch.stor…`	LaTeX configuration
L6079	69	`\begin{NiceTabular} {cc}[hvlines,arraystretch=2]`	text table (`{NiceTabular}` , `{tabular}`)
L6132	69	`\ExplSyntaxOn`	LaTeX configuration
L6164	70	`\begin{NiceTabular}{ccc}[hvlines]`	text table (`{NiceTabular}` , `{tabular}`)
L6352	72	`\usepackage[spanish,es-noshorthands] {babel}`	LaTeX configuration
L6371	72	`\let\TeXyear\year`	LaTeX configuration
L6432	72	`\DocumentMetadata{tagging = on}`	text table (`{NiceTabular}` , `{tabular}`)
L6488	73	`\ExplSyntaxOn`	LaTeX configuration
L6510	73	`\NiceMatrixOptions`	text table (`{NiceTabular}` , `{tabular}`)
L6525	73	`\begin{NiceTabular}{@{}llr@{}}`	text table (`{NiceTabular}` , `{tabular}`)
L6561	74	`\ttfamily \small`	text table (`{NiceTabular}` , `{tabular}`)
L6592	74	`\begin{NiceTabular}{ccc}[hvlines]`	text table (`{NiceTabular}` , `{tabular}`)
L6619	75	`\tcbset`	text table (`{NiceTabular}` , `{tabular}`)
L6720	76	`\begin{NiceTabular}{{>{\columncolor{red!15}}cllr}[vline…`	text table (`{NiceTabular}` , `{tabular}`)
L6948	80	`\NiceMatrixOptions{xdots/horizontal-labels}`	TikZ decoration
L6967	80	`$\begin{pNiceArray}{ccc|cc}[first-row,last-col,margin]`	calligraphic braces (TikZ decoration); the braces of the math font are shown for p. 79
L7443	86	`\pgfset`	text table (`{NiceTabular}` , `{tabular}`)